



Advanced IDS: a comparative study of datasets and machine learning algorithms for network flow-based intrusion detection systems

Jose Carlos Mondragon¹ · Paula Branco³ · Guy-Vincent Jourdan³ · Andres Eduardo Gutierrez-Rodriguez^{2,4} · Rajesh Roshan Biswal¹

Accepted: 1 March 2025 / Published online: 1 April 2025
© The Author(s) 2025

Abstract

Globally, cyberattacks are growing and mutating each month. Intelligent Intrusion Network Detection Systems are developed to analyze and detect anomalous traffic to face these threats. A way to address this is by using network flows, an aggregated version of communications between devices. Network Flow datasets are used to train Artificial Intelligence (AI) models to classify specific attacks. Training these models requires threat samples usually generated synthetically in labs as capturing them on operational network is a challenging task. As threats are fast-evolving, new network flows are continuously developed and shared. However, using old datasets is still a popular procedure when testing models, hindering a more comprehensive characterization of the advantages and opportunities of recent solutions on new attacks. Moreover, a standardized benchmark is missing rendering a poor comparison between the models produced by algorithms. To address these gaps, we present a benchmark with fourteen recent and preprocessed datasets and study seven categories of algorithms for Network Intrusion Detection based on Network Flows. We provide a centralized source of pre-processed datasets to researchers for easy download. All dataset are also provided with a train, validation and test split to allow a straightforward and fair comparison between existing and new solutions. We selected open state-of-the-art publicly available algorithms, representatives of diverse approaches. We carried out an experimental comparison using the Macro F1 score of these algorithms. Our results highlight each model operation on dataset scenarios and provide guidance on competitive solutions. Finally, we discuss the main characteristics of the models and benchmarks, focusing on practical implications and recommendations for practitioners and researchers.

Keywords Intrusion detection system · Network flows · Cybersecurity benchmarks

1 Introduction

Network intrusion detection applications have developed in recent years due to the increasing amount of global cybersecurity threats. During 2022, the first quarter had 4.5 times more attacks than the same period in 2021 [1, 2]. For the second quarter of 2022, 60 million attack attempts were blocked, which means a 74.6% increase over the previous months [3]. Intrusion detection systems are needed to protect the service of applications and avoid attacks such as the MyFitnessPal leak that hit more than 150 million users [4]. Thousands of attacks are performed every day on networks and many of them are not stopped or even identified. These attempts are evolving as technology implemented for online services is advancing toward faster and more complex anti-threat sys-

tems [5, 6]. Different settings such as Internet of Things (IoT) networks, cloud-based services, high-performance clusters, or traditional managed servers are vulnerable to specific attacks, and a suitable solution for one configuration could become a less suitable option for another.

One option to alert on attacks is through the use of Network Intrusion Detection Systems (NIDS). This software analyzes the packets of a network searching for certain patterns or applying defined rules to warn the administrator. As the task involves predicting the type on communication between end point, machine learning (ML) models can be trained to identify normal traffic and threats.

Training a model packet by packet is time consuming and does not protect user privacy [7]. Instead, recent systems use specialized software known as Packet Capture (PCAP) systems to translate an entire session to a summarized version known as network flow. This aggregated view can be seen as

Extended author information available on the last page of the article

a dataset row, generating a compressed perspective of communication between endpoints. Threats are assumed to have distinctive network flow patterns that the model recognizes.

To generate Intelligent NIDS there are two main components: an algorithm to train a model and network flow datasets showcasing threats mixed with normal traffic. Recent solutions propose a variety of methods ranging from shallow ML such as Gradient Boosting [8] techniques to Deep Learning Graph Based Neural Networks [9]. We must highlight that, the algorithm alone is not enough to generate an effective NIDS, the algorithm needs to be trained with tailored network flows that describe different attack patterns specific to the network use case.

IoT networks, software defined networks, or corporative networks have structures and traffic with different packets and patterns. A PCAP software can work on alternative cases but the generated network flows will vary between networks. The ability of an algorithm to adapt the model to more than one scenario is a major milestone for its operational implementation. Unfortunately, network traffic data is typically private and companies cannot share this data with researchers. Thus, new algorithms are usually trained on datasets mainly generated on lab scenarios in which network devices log packets for predefined attacks [10, 11], classifying the network flows with rules. Although this is a viable solution, the lab setting is assembled just for the test rendering designed traffic and not thorough real threats mechanics. Efforts have been made to acquire data from operational networks such as Litnet [12] but this setting requires an additional task of manual or rule-based threat classification that may have errors.

1.1 Research gap

Currently, there is no single dataset capable of testing a solution thoroughly. Different benchmarks are designed with specific sets of attacks. Network traffic and architectures propose different scenarios even if the focused threats are similar. Although different techniques should be tested on a wide range of possibilities, recently proposed ones are studied on no more than 4 different benchmarks [13], which limits the conclusions. Other commonly used datasets are outdated and do not represent current real threats. One of the most studied datasets, the NSL-KDD [14], is an updated version of KDD99 [15] and was published in 2009 when attacks had different characteristics when compared to attacks in 2023. More recent datasets such as CIC IDS 2017 from New Brunswick University, are updated and freely available but have building issues that need to be tackled [16]. Other benchmarks such as Litnet [12] or Hikari [17] are new and consider recent threats but are still finding their way as potential scenarios in current research studies.

A few recent surveys report the available datasets [2, 4], and new methods use some of these datasets. However, in the experimental setting of the papers the benchmarks are modified, compromising a fair comparison to other studies. To the best of our knowledge, there is no recent study that compiles datasets from the previous 11 years and tests different state-of-the-art techniques.

As proposals are tested on different and outdated sets, even with alternative subsamples or reconditioned data, a comparison that highlights the advantages and opportunities of recent techniques is needed. The characteristics of algorithms, their performance, and implementation details are important for practitioners and researchers not only to solve specific network scenarios, but to develop new systems on a common ground.

1.2 Contribution

This paper focuses on the comparison of state-of-the-art ML algorithms on multiple recent datasets. By considering 7 different algorithms on 14 datasets, we propose a common unified baseline for all of them. Training and testing with the same data allows a wider perspective of the different characteristics of datasets and solutions along with advantages and opportunities. To assess the performance of each solution the Macro F1 score was selected. Characteristics such as imbalance ratio, attribute types, and sample count are shown on every dataset along with implementation and usage details. The objective is not to identify one best method, but to outline the aspects of different settings that researchers and practitioners could use. The contributions of this paper are as follows:

- Study 14 recent Network Flow datasets to be used on Intelligent Intrusion Detection Systems training highlighting key characteristics such as size, imbalance and threats.
- Compile and make available the first benchmark of 14 network flow datasets, providing the used train, validation, and test sets to allow the direct comparison of new algorithms (<https://github.com/UOttawa-Cyber-Range-Research/GuardiansOfTheNetwork>).
- Analysis of 7 different freely available state-of-the-art algorithms identifying their core technique, implementation details, and usage challenges.
- Comparison of 7 algorithms on the 14 datasets, in a uniquely broad study, highlighting the performance scores but also the characteristics and challenges of the methods such as robustness to imbalance, memory requirements, volume of ingested instances, and types of attributes.

1.3 Impact

The benchmark data was prepared using recent publicly available network flow datasets. This allows the direct comparison of new algorithms against existing ones in different contexts. The benchmark also opens the possibility to merge datasets and test more than one scenario simultaneously. Whether is for corporative networks, web servers or IoT, testing the model in alternative use cases allows researchers to measure the performance and adaptability of their solutions. Moreover, it allows researchers to identify the stronger aspects of their solutions as well as any less desired behavior that might be associated with specific deployment settings.

The freely available algorithms listed on this study allow practitioners to start with solutions tested on a common ground in different scenarios that might resemble their own infrastructures. By obtaining the datasets and code, they benefit from a quick start on compatible settings. The variety of the categories of the algorithms presented serves as a starting point for new techniques. In practice, this allows researchers to quickly deploy and test a particular type of algorithm, such as a neural network or a graph-based solution, that they have not considered before and are willing to try.

1.4 Organization

This paper is organized as follows: Section 2 introduces common background and theory behind the Network Flow Intrusion Detection Systems. Section 3 describes the datasets and algorithms used in this study highlighting the characteristics of the datasets while adding relevant information regarding usage challenges and implementation details. Section 4 presents the experimental settings and the results of the study. Section 5 discusses the results obtained. Finally, Section 6 provides the conclusions and discusses the future research directions of this paper.

2 Related works

IDS have evolved in recent years due to different technologies. ML covers a set of techniques that searches for patterns in sets of data, generalizing the findings and providing a method to apply them in future observations [18]. The implementation of these models has yielded progress for IDS and has allowed their adaptation to old and new attacks [1]. Understanding these methods' taxonomy as well as the used technologies allows the development of enhancements and new solutions.

There are two main perspectives on Intrusion Detection: Host and Network Intrusion Detection Systems. Host Intrusion Detection Systems (HIDS) are focused on the identification of irregular operations on devices such as com-

puters or routers from an individual perspective [19]. The solutions designed for HIDS study application logs and memory footprints. Network Intrusion Detection Systems (NIDS) are designed to work externally to hosts identifying erroneous patterns in the communication between devices, especially with servers and sensitive equipment [19, 20].

An initial approach was signature-based recognizers [21]. NIDS based on this idea are loaded with rules to identify common packets produced to perform specific attacks. Although achieving effectiveness and low false positive rates, they lack adaptation to both, zero-day flaws and small variations on previous attacks [22]. To overcome this challenge, researchers looked for ML solutions that could generalize patterns in data.

NIDSs study two types of network traffic: packets and flows. While the former is the specific real-time communication, the latter tries to summarize the connection between two devices. As the internet grows, classifying each packet on the networks requires more hardware and implies a tedious labeling task [23]. Not every packet is, by itself, an identifiable threat. Due to the limited analysis capacity and the categorization problem, Network Flows datasets and techniques for this setting started to be developed more recently [23]. In this paper, we focus on systems that use Network flows to study and classify multiple attacks.

Not every attack should be treated equally and specific reports allow administrators to apply adequate policies. NIDSs have implemented new techniques to not only detect the intrusion, but to predict what type of attack is represented on the network, and eventually forward the specific details to Intrusion Prevention Systems that introduce measures to prevent it. Figure 1 shows the cycle in which the flows are not only identified but stored to train the system again.

Multi-class Network Flows IDS allow threat-specific identification. To train these intelligent systems, a packet capture extractor first records the communication between two devices. When the transmission has ended, it extracts and aggregates features from the data transfer and processes the register with the analysis module [24]. Finally, the NIDS compares the pattern to the developed model and decides to report a specific alert.

Methods from different ML areas have been implemented for this task. Shallow ML as well as Deep Learning (DL) techniques display promising results [20, 21, 25]. Nevertheless, attackers are varying and renewing their own procedures on a day-to-day basis, requiring the solutions to be trained and deployed continuously with updated scenarios, as Fig. 1 shows. The following subsections introduce representative recent techniques and datasets used, analyzing characteristics that open the perspective of researchers and practitioners in the field. After this analysis, the techniques are tested on different scenarios to finally discuss their characteristics and opportunities.

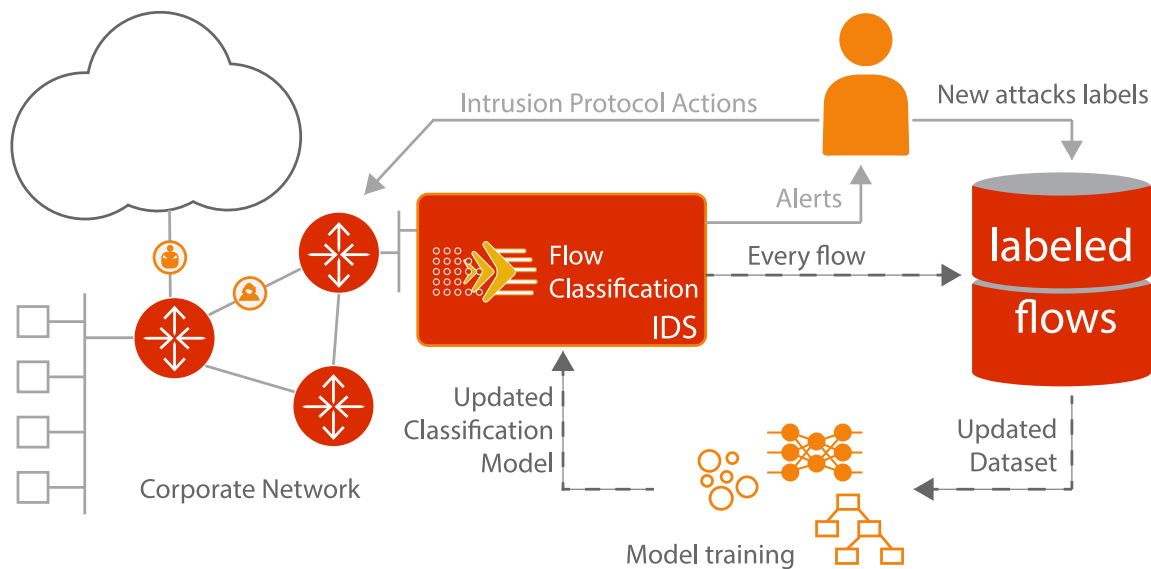


Fig. 1 Flow Based NIDS operation: The network packets are captured to build each flow. When a threat is detected an alert is issued to Systems Managers and Intrusion Prevention Systems. The flows are stored for new model training and updates

3 Materials and methods

On a general basis, ML methods for intrusion detection deal with the task of classification. Given a training dataset with records containing categorical and numerical features, an algorithm will try to identify patterns to predict a category. On a supervised training task such as the ones presented in this paper, each record is annotated with a label. The set of these labels constitutes the different classes that the predictor should output. In the testing stage, the model will be presented with unseen samples to evaluate a certain metric [20]. We considered a set of performance assessment metrics that are suitable for our imbalanced multi-class setting. A description of these metrics can be found in Section 3.6. In this section, we present the datasets and their respective preprocessing along with the algorithms and the necessary modifications applied to have them operate on the datasets.

3.1 Datasets

In ML research, new methods must be trained and tested with representative data that previous studies have found as benchmarks in a certain field. This brings standardized datasets that researchers around the globe can download and use to train their methods. Having a common set of files allows a fast initial setup as well as a baseline for performance metrics. Although this has been tried in network flow datasets, it is still challenging to find a single source of datasets, or even a single dataset that is treated in the same way throughout different methods. Surveys such as [18, 21, 23, 26, 27] and [28] have pointed out many of them, unfortunately, some of them have already become outdated or have even been discontin-

ued, rendering them inaccessible and not standard anymore. Different from other ML fields, in which common sets can be valid for a long time, cybersecurity datasets become old as new attacks and architectures are developed, which has been faster in recent years [1, 3, 27].

The lab process to develop one of these benchmarks involves the setup of a network, close to a desired scenario. Then, certain devices must be infected with the specific software that will create the malicious traffic while others should resemble normal, daily communications. Once the test starts, the research team should be capable of starting and ending the attacks while maintaining the time and attacker IP address log. On the network, a device must log all the traffic for later analysis. Once the tests have ended, the PCAP extractor software analyzes the packets and outputs a set of files with the summary of every communication that has happened on the devices. As a final step, the information about the attacks regarding the IP address, start time, and end time is used to label each of the flows. This process, though large, is needed to assure the correct labeling of the dataset and is depicted in Fig. 2.

In this section, a collection of datasets is presented along with their characteristics and some considerations that practitioners should account for when using them. All of these have been featured in at least one paper in the previous 11 years and have been confirmed to be publicly accessible. We expect this to be a good starting point for future research while allowing a wider perspective.

During the different characterizations, we introduced a change on every dataset, although this is only representative for categorical features enabled methods. We considered that the network port cannot be normalized as a numeric attribute

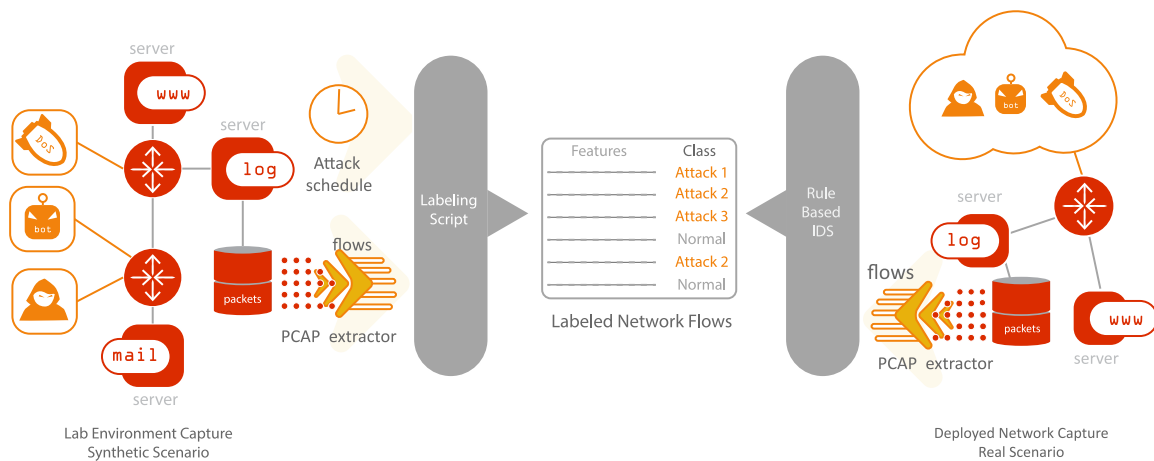


Fig. 2 Labeling Process: Laboratory-based scenarios are processed by a Labeling script considering the Attack Schedule planned by researchers. Real network scenarios are labeled by another Rule based IDS

but as a part of the qualitative characteristics of each dataset. A problem with this approach is the 65535 values that the feature could get. In this regard, we transformed the attribute to a representation that stated if the connection was done to (i) Well-Known (0-1024), (ii) Dynamic (1025-49151), or (iii) Ephemeral ports (49152-65535) [29].

On top of the dataset size, either instance or feature wise, the data points distribution in terms of class is important to better understand the difficulty of each scenario. While Imbalance Ratio (IR) [30] displays the quotient between the most represented class and the least represented one, it fails to show the general distribution of the rest of the classes. Regarding this, we present the Average Imbalance Ratio ($AvIR$), as the mean of all the quotients between the number of majority class examples and the number of examples of each class except for the majority class. This average does

not consider the majority class divided by itself. The Imbalance Ratio with respect to a class and the Average Imbalance Ratio are formally presented in (1) and (2):

$$IR = \frac{N_{c_{maj}}}{N_{c_{min}}} \quad (1)$$

$$AvIR = \frac{\sum_{c_i \in C \wedge c_i \neq c_{maj}} \frac{N_{c_{maj}}}{N_{c_i}}}{|C| - 1} \quad (2)$$

where C is the set of classes, c_{maj} is the majority class, c_{min} is the minority class, and N_{c_i} is the number of instances from a given class c_i . With these characteristics, a wider perspective of each dataset is shown in the summary of the datasets in Table 1.

Table 1 Summary of main characteristics of the datasets used including the total number of instances, the total number of classes, the Imbalance Ratio with respect to the least represented class (IR), and the Average Imbalance Ratio ($AvIR$)

Dataset	Year	Instances	Classes	IR	$AvIR$
OPCUA [31]	2020	107 634	4	12 335.334	4 615.994
InSDN [32]	2020	343 888	8	7 173.059	1 234.830
Hikari [17]	2021	555 278	6	105.956	49.883
UNSW-NB15* [33]	2021	2 390 275	9	13 995.256	2 296.324
CIC IDS 2017 [10]	2017	2 097 863	16	145 188.727	28 513.965
CIDDS [34]	2017	9 884 894	4	3 919.003	1 548.610
NDSec-1 [35]	2017	1 384 626	27	915 613.000	243 800.586
ISCX-2012 [36]	2012	1 723 818	5	458.989	246.957
ToN IoT* [33]	2021	16 940 365	10	1 780.841	329.728
USB IDS [37]	2021	4 813 395	17	480.421	132.318
Litnet [12]	2020	38 988 542	13	75 077.333	13 780.948
Bot IoT* [33]	2021	37 643 287	5	7 506.866	1 912.446
DDoS 2019 [38]	2019	48 269 665	14	39 519.773	3 748.479
CSE CIC 2018 [39]	2018	62 999 332	5	1 524 716.872	231 255.364

* Using Netflow versions [33]

Table 2 OPCUA dataset: Distribution of instances according to the label

Class	Instances	Proportion
Normal	33 666	31.19%
DoS	74 012	68.76%
MITM	6	0.01%
Impersonation	49	0.05%
Total	107633	100.0%

Table shows the class, the instances associated to it and the ratio according to the total amount of examples

3.1.1 OPCUA

As a product of the observations of secure industrial communications for OPCUA protocol deployments, IEEE Dataport features the database [31]. The file is a compilation of TShark¹ labeled traffic of a testbed done in a lab setting. It includes 3 different types of attacks: Denial Of Service (DoS), Eavesdropping/Man in the middle (MITM), and Impersonation (Imp). All the attacks were generated by a script from a single node and produced a dataset of 107 634 records, with the distribution as shown in Table 2. It includes 30 features, one binary label column indicating if the record corresponds to an attack, and one multi-class column stating the type of the attack. The attributes are distributed in 23 numerical and 7 categorical columns. The dataset could avoid 3 numerical (service_errors, status_errors, f_flowStart) features and 1 category (proto) as they have the same value for every instance. To obtain this dataset, a direct CSV (comma separated values) file link is provided from the site².

3.1.2 InSDN

One of the new scenarios that telecom carriers and providers are supporting is the Software Defined Network (SDN) [40]. InSDN is the first attempt to develop a dataset that gathers information and provides a benchmark for these structures. Elsayed et al. [32] argue that these types of networks could become compromised if the attacker gains control of the main controller. Therefore, IDS are desired and even needed to detect abnormal operations. By being an update to other datasets that only addressed the Denial of Service attacks, they do not display behaviors on different Network Layers. This dataset includes the following attacks: Denial Of Service (DoS), Distributed Denial Of Service (DDoS), Brute Force Attacks (BFA), Web Application attacks (Web), Exploits (U2R), Probes, and Botnet traffic (Botnet). Is important to highlight that SDNs are able to be simulated without the

Table 3 InSDN: Distribution of instances according to the label. Table shows the class, the instances associated to it and the ratio according to the total amount of examples

Class	Instances	Proportion
Normal	68 423	19.897%
DoS	53 616	15.591%
DDoS	12 1942	35.460%
Probe	98 129	28.535%
BFA	1405	0.409%
Web-Attack	192	0.056%
BOTNET	164	0.048%
U2R	17	0.005%
Total	343 888	100.0%

actual network devices such as routers or switches as it was done by the researchers [32]. The authors report that once the attacks were generated, CICFlowMeter [41] was used to extract 84 attributes distributed in 5 categorical, 1 temporal, 75 numerical, and one as a multi-class label. By generating the traffic from different virtual machines they could report 343 939 instances. However, when we downloaded the dataset, joined the records, and eliminated duplicates the number of instances was downsized to 343 888. The files are public and downloadable from their site as a compressed archive³. Once it decompresses, 3 CSV files are displayed with one of them having only normal traffic, a second one with attack traffic, and a final one with metasploitable 2 (vulnerable Linux OS image) attack data. Table 3 provides the main details regarding the class distribution in this dataset.

3.1.3 Hikari

Encrypted data has not been the focus of the dataset generation on cybersecurity network flows. Ferriyan et al. [17] identified this gap and produced Hikari Dataset. Published in 2021, the authors claim that design requirements for this set included having a complete capture of network communications, anonymity, ground-truth labels, updated attacks, and encrypted information. By analyzing other 12 datasets such as CIC IDS 2017 [10], UGR 16 [42], UNSW-NB15 [11], CAIDA[43], and MAWILab [44] among others, they could identify specific scenarios. Simulating application layers attacks, particularly HTTPS (Hyper-Text Transfer Protocol Secure), they implemented two networks, an attack and a victim separated by a router. The services running on victim computers were HTTP (Hyper-Text Transfer Protocol)

¹ <https://www.wireshark.org/docs/man-pages/tshark.html>

² <https://digi2-feup.github.io/OPCUADataset/>

³ <https://aseados.ucd.ie/datasets/SDN/>

Table 4 Hikari: Distribution of instances according to the label

Class	Instances	Proportion
Background	171 151	30.64%
Benign	347 431	62.57%
Bruteforce	5 884	1.06%
Bruteforce-XML	5 145	0.93%
Probing	23 388	4.21%
XMRIGCC CryptoMiner	3279	0.59%
Total	555 278	100.0%

Table shows the class, the instances associated to it and the ratio according to the total amount of examples

Servers with Wordpress⁴, Drupal⁵ and Joomla Applications⁶. After two days, normal traffic as well as brute force attacks had already been logged. Different from other datasets, this one has a Background Traffic label, which authors indicate could have malicious traffic. Zeek⁷ was used to validate this idea and they found crypto mining attacks. Authors claim this is due to Zeek rules and that would be considered part of the dataset. Finally, they applied the CICFlowMeter [41] and scripts to label the dataset according to planned attacks. Hikari [17] contains 555 278 as reported by the authors and confirmed after we downloaded the file. The downloadable CSV dataset⁸ includes 86 features that are distributed over 4 categorical features, 1 multi-class label and 81 numeric. No columns were found with the same value for every instance. Table 4 provides the main details regarding the class distribution in this dataset.

3.1.4 UNSW-NB15

The generation of cybersecurity datasets will continue as each one will become outdated. Implementation of new attacks and network structure was the objective of UNSW-NB15 [11]. With critiques over the well known NSL-KDD [14] and KDD99 [15], this dataset was generated based on a 3 server topology and instead of installing malicious software, Ixia Traffic Generator is used to mimic the attacks based on cyber-security standards. After 31 hours of attack generation, the stored PCAP files were analyzed by Argus [45] and Bro-IDS [46] software to finally label the flows. The process ended with a set of 49 features for 1 964 509 records including Normal traffic as well as Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, ShellCode, and Worms attacks.

⁴ <https://wordpress.org/download/>

⁵ <https://www.drupal.org/download>

⁶ <https://downloads.joomla.org/>

⁷ <https://zeek.org/>

⁸ <https://zenodo.org/record/5199540>

Table 5 UNSW-NB15: Distribution of instances according to the label

Class	Instances	Proportion
Benign	2 295 222	96.02%
Fuzzers	22 310	0.93%
Analysis	2 299	0.10%
Backdoor	2 169	0.09%
Dos	5 794	0.24%
Generic	31 551	1.32%
Reconnaissance	16 560	0.69%
Shellcode	1 427	0.06%
Worms	164	0.01%
Total	2 390 275	100.0%

Table shows the class, the instances associated to it and the ratio according to the total amount of examples

As with other datasets, files are available for download⁹. Nevertheless, information is divided into 4 different sets, 3 description documents, and a 2 file training-test split. While this poses a first challenge, the general understanding is a reasonable process to get the final compiled data. Features vary from other benchmarks as some of these were specifically designed for the dataset and the usage of the PCAP analyzers. Due to the differences to other files, a second dataset was generated by reading the generated PCAP files and exporting them in the NetFlow Standard [33, 47]. This second version, also available for download¹⁰, is a dataset with 43 features distributed over 6 categorical, 37 numerical, one binary attack tag, and a multi-class label. The authors of this alternate proposal state, are trying to avoid specific attribute picking due to different PCAP extractors and procedures. When deciding which features to include for a future classification stage, the actual evaluation of a NIDS may become biased by the dataset researchers' domain knowledge. The research adds a comparative study training different classifiers on original and new versions. Reported results were better on the NetFlow [47] standard versions than on the previous ones with ranges between 1% and 3% on the F1-score. This paper uses the second NetFlow-based version. Table 5 provides the main details regarding the class distribution in this dataset.

3.1.5 CIC IDS 2017

Gharib et al. [26] identified 11 criteria for the development of cyber-security benchmarks based on analysis of 11 datasets. The research included the well-known KDD99 [15] and introduced NSL-KDD [14] as an update with critiques over the redundancy of records. It also added the ADFA [48] dataset as one of the newest with issues on the attack separa-

⁹ <https://research.unsw.edu.au/projects/unsw-nb15-dataset>

¹⁰ https://staff.itee.uq.edu.au/marius/NIDS_datasets/

Table 6 CIC IDS 2017: Distribution of instances according to the label

Class	Instances	Proportion
BENIGN	1 597 081	76.129%
Botnet	736	0.035%
DDoS	95 144	4.535%
DoS GoldenEye	7 567	0.361%
DoS Hulk	158 449	7.553%
DoS Slowhttptest	1 741	0.083%
DoS Slowloris	3 998	0.191%
FTP-Patator	3 972	0.189%
Heartbleed	11	0.001%
Infiltration	36	0.002%
Infiltration - PortScan*	67 023	3.195%
Portscan	159 040	7.581%
SSH-Patator	2 961	0.141%
Web Attack - Brute Force	73	0.003%
Web Attack - SQLInjection	13	0.001%
Web Attack - XSS	18	0.001%
Total	2 097 863	100.0%

Table shows the class, the instances associated to it and the ratio according to the total amount of examples. *This label was created as the network flow was not distinguishable

tion and lack of attack diversity. Due to all the issues found, Sharaldin et al. [10] created the Intrusion Detection Dataset that has been featured in at least 14 papers as reported by Chou and Jiang [49] and cited by more than 2 250 papers [50]. This benchmark is the result of a 5-day log with 7 types of attacks for a total of 16 labels. The experiment was set up in two networks with 4 attackers including Linux and Windows Machines. The victim network represented a common company architecture with 3 servers including Windows and Linux operating systems as well as 7 clients with Mac OS, Linux, and Windows. During each of the record days benign and attack specific traffic was generated according to a specific schedule. The final features were extracted with CICFlowMeter [41] and labeled for a final dataset of 85 attributes distributed on 5 categorical, 1 timestamp, 78 numeric, and one multi-class label. The 5 files (one for each day) are downloadable from the University of New Brunswick site¹¹.

Due to the cyber-security need for datasets, each benchmark is studied for its validity and acceptance. Several problems have been reported by Engelen et al. [16], Rosay et al. [51], Liu et al. [52] and Lanvin et al. [53]. It comes to our attention that the original CIC IDS 2017 [54] download site does not report on these issues nor gives a statement on the validity of their methodology. The main correction regarding the dataset comes from a fix on the CICFlowMeter [41],

¹¹ <https://www.unb.ca/cic/datasets/ids-2017.html>

Table 7 CIDDS: Distribution of instances according to the label

Class	Instances	Proportion
Benign	9 555 528	96.67%
Scan	313 197	3.17%
Bruteforce	2 438	0.02%
Portscan	13 731	0.14%
Total	9 884 894	100.0%

Table shows the class, the instances associated to it and the ratio according to the total amount of examples

version 3, as its analysis was terminating communications before it should. A second correction is done on connections that were attempted but did not result in any actual attack. These changes generated different network flows, hence a new version of the dataset. For example, the original version had 2 828 164 flows, while this newer has 2 097 863. In this paper, the versions provided on WTMC¹² updated site were used. Table 6 provides the main details regarding the class distribution in this dataset.

3.1.6 CIDDS

Being able to resemble common networks or scenarios is a main concern for dataset design. The objective is to train models and agents capable of recognizing attacks in similar situations. The CIDDS-1 [55] dataset was built just to meet these general guidelines. By simulating regular work hours and common server workloads this benchmark introduces internal as well as external attacks. An OpenStack architecture with Windows and Linux clients adds a layer of heterogeneity to traffic logs which produce Denial Of Service, Brute Force, and Port Scanning samples. The second version CIDDS-2[23] was generated having the same original architecture but adding more Portscan attacks and Benign communications. The files are downloadable from their site¹³. In this paper we used a joint set of CIDDS-1 and CIDDS-2. Table 7 provides the main details regarding the class distribution of the complete dataset.

3.1.7 NDSec-1

A thorough analysis of existing datasets brings opportunities to light. NDSec-1 is a dataset designed for attacks that would come from real exploitation tools and, up to the creation date, recent attack reports [35]. By focusing on the malicious traffic rather than on the normal one, the benchmark introduces

¹² <https://intrusion-detection.distrinet-research.be/WTMC2021/>

¹³ <https://www.hs-coburg.de/forschung/forschungsprojekte-oeffentlich/informationstechnologie/cidds-coburg-intrusion-detection-data-sets.html>

realistic scenarios of attacks within two subnets, one simulating an internal corporate network and another one simulating the internet. The traffic was collected on the company simulated intranet with attack servers coming from the external. Linux and Windows computers were deployed in the infrastructure with a specific compromised BYOD (Bring Your Own Device) policy device. The report done for this dataset includes the specific tools that were used in the attacks a gives more information on how to obtain them. Even a common Network Intrusion Detector was implemented on the architecture to report some of the attacks. Most of them were not stopped. The authors conclude that “incorporating penetration test suites, recent malware instances, and classic attack tools” can highlight vulnerabilities of IDS. With this

Table 8 NDsec-1: Distribution of instances according to the label

Class	Instances	Proportion
BOTNET CC Command Execution	15	0.0011%
BOTNET CC Communication	65	0.0047%
BOTNET CC Malicious Download	12	0.0009%
BRUTEFORCE FTP-Bruteforce	60	0.0043%
BRUTEFORCE HTTP-Bruteforce	2 126	0.1535%
BRUTEFORCE SSH-Bruteforce	970	0.0701%
DOS DHCP-Starvation	1	0.0001%
DOS DNS-Amplification	2	0.0001%
DOS HTTP-Flood	16 735	1.2086%
DOS SYN-Flood	436 838	31.5492%
DOS UDP-Flood	915 613	66.1271%
EXPLOIT Malware Infection	8	0.0006%
MALWARE Command Execution	2	0.0001%
MALWARE Communication	25	0.0018%
MALWARE Malicious Download	3	0.0002%
MISC Malicious FTP-Upload (SFTP)	3	0.0002%
MISC SSL-Proxy (MitM)	46	0.0033%
NORMAL	9 958	0.7192%
PROBE IP-Scan/Host-Scan	4	0.0003%
PROBE Port-Scan	2000	0.1444%
PROBE Vulnerability-Scan	98	0.0071%
SPOOFING ARP-Spoofing	11	0.0008%
SPOOFING DNS-Spoofing	1	0.0001%
WEBATTACK SQL-Injection	26	0.0019%
WEBATTACK XSS Infection	1	0.0001%
WEBATTACK XSS Reflected	1	0.0001%
WEBATTACK XSS Stored	2	0.0001%
Total	1 384 626	100.0%

Table shows the class, the instances associated to it and the ratio according to the total amount of examples

Table 9 ISCX-2012: Distribution of instances according to the label

Class	Instances	Proportion
Normal	1 667 510	96.734%
HTTPDoS	3 633	0.211%
Infiltration	10 127	0.587%
BruteForceSSH	5 219	0.303%
IRCDoS	37 329	2.165%
Total	1 723 818	100.0%

Table shows the class, the instances associated to it and the ratio according to the total amount of examples

information, the 4 downloadable files¹⁴ include 23 features distributed on 2 timestamps, 10 categorical, 7 numeric, one binary label, two multi-class labels providing details and a feature with comments. In this paper, we used the labels given by the concatenation of the two details attributes. Table 8 provides the main details regarding the class distribution in this dataset.

3.1.8 ISCX-2012

After tests on datasets such as NSL-KDD [14] and KDD99 [15], it has been a continuous attempt to generate realistic traffic and updated attacks. ISCX-2012 [36] is a dataset generated on a 17 Windows computer testbed with 3 main servers for Email, Apache HTTP, IIS (Internet Information Services) HTTP, DNS (Domain Name Server), SSH (Secure Shell), FTP (File Transfer Protocol), and NAT (Network Address Translation) services. Designed on 4 specific scenarios this benchmark generated 2 450 324 flows with Normal, HTTP-DoS, Infiltration, BruteForceSSH, and IRCDoS (Internet Relay Service DoS) attacks. The 19 features are distributed on 12 categorical, 2 timestamps, 4 numerical, and one multi-class category. The 12 downloadable files¹⁵ have to be joined to integrate one complete dataset with 1 723 818 instances after duplicate removal. Table 9 provides the main details regarding the class distribution in this dataset.

3.1.9 ToN IoT

Internet-enabled devices such as sensors, Smart TVs, and even cell phones are changing the way to offer services and connections on networks. IoT is quickly becoming a complex structure of millions of interconnections with different computation centers. ToN IoT [56] dataset was designed to display phenomena happening in real scenar-

¹⁴ <https://www2.hs-fulda.de/NDsec/NDsec-1/Files/>

¹⁵ <https://www.unb.ca/cic/datasets/ids.html>

Table 10 ToN IoT Distribution of instances according to the label

Class	Instances	Proportion
Benign	6 099 379	36.01%
backdoor	16 809	0.10%
ddos	2 026 234	11.96%
dos	712 609	4.21%
injection	684 465	4.04%
mitm	7 682	0.05%
password	1 153 323	6.81%
ransomware	3 425	0.02%
scanning	3 781 419	22.32%
xss	2 455 020	14.49%
Total	16 940 365	100.0%

Table shows the class, the instances associated to it and the ratio according to the total amount of examples

ios by deploying a network on three layers: Cloud, Fog, and Edge. Considering data flows from servers as well as user computers or phones and sensors, data was generated comprising 9 different attacks. The network was simulated using Software Designed Network(SDN) equipment including scenarios such as Azure connections and Queue brokers. Downloading the 23 from the site¹⁶ after browsing through the different versions explained in the paper, renders a final dataset with 22 339 021 records for 9 attacks. The 44 features are designed taking information from the Zeek [57] tool and are distributed on 1 timestamp, 28 categorical, 13 numerical, 1 binary label, and 1 multi-class label. PCAP files are also available for this dataset and Sarhan et al. [33] also generated an alternate version trying to avoid very specific domain knowledge from the creator. This 16 940 365 instances version includes 43 features distributed over 6 categorical, 37 numerical, one binary attack tag, and a multi-class label. Results are enhanced as authors compared models trained on both datasets with a Macro F1 score of 91.6% as opposed to 67.9%. In this paper, we used the alternate version. Table 10 provides the main details regarding the class distribution in this dataset.

3.1.10 USB IDS

Different critiques over datasets such as the widely adopted CIC IDS 2017 [10] have been put forward regarding the distance to actual scenarios. One of these is the network-only view, which affects the quality of the data at the application layer. In a discussion in [37], Catillo et al. propose a different perspective regarding dataset design. With a multi-layer approach and specific server settings, including even some attack protection, they claim that a closer-to-reality bench-

¹⁶ <https://research.unsw.edu.au/projects/toniot-datasets>

Table 11 USB IDS: Distribution of instances according to the label

Class	Instances	Proportion
BENIGN	304 739	6.33%
Hulk-Evasive	756 985	15.73%
Hulk-NoDefense	851 130	17.68%
Hulk-Reqttimeout	855 630	17.78%
Hulk-Security2	813 046	16.89%
Slowhttptest-Evasive	6 690	0.14%
Slowhttptest-NoDefense	6 690	0.14%
Slowhttptest-Reqttimeout	5 754	0.12%
Slowhttptest-Security2	6 687	0.14%
Slowloris-Evasive	1 781	0.04%
Slowloris-NoDefense	1 782	0.04%
Slowloris-Reqttimeout	6 781	0.14%
Slowloris-Security2	1 781	0.04%
TCPFlood-Evasive	302 571	6.29%
TCPFlood-NoDefense	291 720	6.06%
TCPFlood-Reqttimeout	302 685	6.29%
TCPFlood-Security2	296 943	6.17%
Total	4 813 395	100.0%

Table shows the class, the instances associated to it and the ratio according to the total amount of examples

mark can be conceived. The proposed network architecture is oriented towards different DoS attacks and is comprised of an attacker, client, and victim node for a specific Apache HTTP service. It is important to highlight that server configuration was tailored to mimic those of industry implementations. After the attacks were performed, 4 813 395 instances were labeled with 16 different classes. CICFlowMeter [41] tool was used to finally label the downloadable files¹⁷ to obtain 84 attributes distributed in 5 categorical, 1 timestamp, 75 numerical, and 1 multi-class label. Table 11 provides the main details regarding the class distribution in this dataset.

3.1.11 Litnet

Realistic scenarios are a goal of researchers on cyber-security datasets. Ideally, the placement of network logging devices could just take that information from deployed networks. Litnet [12] is a benchmark designed and obtained from an operating network, connected to a University. The Lithuanian Research and Education Network served as the sampling field to recover data from attacks and normal traffic. The deployed infrastructure includes links between 5 cities, 2 university campuses, a firewall, and various end users such as schools or municipalities. After gathering the flows, researchers could identify 12 different attacks in a 10-month period between 2019 and 2020. Extracting the features based on Netflow

¹⁷ <https://idsdata.ding.unisannio.it/>

Table 12 Litnet: Distribution of instances according to the label

Class	Instances	Proportion
none	35 811 888	91.852%
http_f	22 940	0.059%
icmp_f	21 089	0.054%
icmp_smf	118 131	0.303%
smtp_b	747	0.002%
tcp_land	52 417	0.134%
tcp_red_w	1 255 702	3.221%
tcp_syn_f	1 580 016	4.053%
tcp_udp_win_p	6 232	0.016%
tcp_w32_w	24 291	0.062%
udp_0	477	0.001%
udp_f	93 436	0.240%
udp_reaper_w	1 176	0.003%
Total	38 988 542	100.0%

Table shows the class, the instances associated to it and the ratio according to the total amount of examples

Standard [47] and labeling them through specific rules, they were able to generate 45 330 333 with 85 features. The files are downloadable¹⁸ from their site with one CSV containing all the records. After duplicated rows and same-value columns removal, a final dataset of 38 988 542 samples with 43 attributes distributed in 31 categorical, 10 numerical, 1 binary label, and 1 multi-class label was obtained. Table 12 provides the main details regarding the class distribution in this dataset.

3.1.12 Bot IoT

As the IoT is gaining more attention due to the vast amount of connected devices, researchers are looking into providing more methods to deploy secure solutions. Bot IoT [58] is a dataset generated with 4 sensors, 1 main server, and 4 machine users connected to the Internet through two interfaces simulating a company network. Attackers, which are on the same LAN (Local Area Network) as the users, carry their operations against the victims while the latter ones are trying to complete their own transactions, either internally or internet based. Authors mirror similar studies but are specific to IoT use cases, obtaining 73 370 443 instances. Recovering the network flows through the Argus tool [45] and labeling with custom scripts, 74 files are available on their site for a complete download. A version divided by type of attack is also available although it contains different features. Although the proposed dataset is accessible¹⁹, Sarhan

¹⁸ <https://dataset.litnet.lt/data.php>

¹⁹ <https://research.unsw.edu.au/projects/bot-iot-dataset>

Table 13 BoT IoT: Distribution of instances according to the label

Class	Instances	Proportion
Benign	135 026	0.36%
DDoS	18 211 658	48.38%
DoS	16 673 183	44.29%
Reconnaissance	2 620 994	6.96%
Theft	2 426	0.01%
Total	37 643 287	100.0%

Table shows the class, the instances associated to it and the ratio according to the total amount of examples

et al. [33] proposed another downloadable version²⁰ based on Netflow Standard [47]. With 37 643 287 instances after duplicate removal, this new dataset includes 43 features distributed over 6 categorical, 37 numerical, one binary attack tag, and a multi-class label. The authors [33] argue that this specific way of extraction is able to avoid specific dataset knowledge and be helpful to generalized solutions. In this paper, we used this alternate version. Table 13 provides the main details regarding the class distribution in this dataset.

3.1.13 DDoS 2019

Specific attacks such as DoS are studied in networks due to their growing importance. Denial of Service(DoS) and Distributed Denial of Service(DDoS) are two attacks that are still simple yet cause great damage in networks [59]. DDoS 2019 [38] is the result of a benchmark that was designed for completeness and diversity, provided that is focused on specific threats. Two networks were deployed to generate the traffic, a victim and an attacker. Having a robust network device infrastructure, they established 4 user computers, 1 Fortinet Firewall, and 1 server. Although the amount of computers in the attacker network is undisclosed, this dataset includes 11 types of DoS attempts generated from it (plus 2 subtypes). The files, downloadable from their site²¹ are available on PCAP and CSV labeled options. Is important to highlight that is not clear if this dataset has the same issue that CIC IDS 2017 reported by Engelen et al. [16], Rosay et al. [51] and Lanvin et al. [53]. For this reason, in this paper, we used the PCAP extractor updated by Engelen et al. [16] and obtained 48 269 665 instances. These contain 85 attributes distributed on 5 categorical, 1 timestamp, 78 numeric, and one multi-class label. Table 14 provides the main details regarding the class distribution in this dataset.

²⁰ https://staff.itee.uq.edu.au/marius/NIDS_datasets/

²¹ <https://www.unb.ca/cic/datasets/ddos-2019.html>

Table 14 DDoS 2019: Distribution of instances according to the label

Class	Instances	Proportion
BENIGN	20 866 440	43.229%
DNS	15 063	0.031%
	2 103 430	4.358%
MSSQL	4 100 933	8.496%
NTP	1 146 273	2.375%
NetBIOS	4 678 755	9.693%
PortMap	7 025	0.015%
SNMP	4 806 875	9.958%
SSDP	2 541 199	5.265%
SYN	316 205	0.655%
TFTP	4 611 036	9.553%
UDP	3 071 489	6.363%
UDP-Lag	4 414	0.009%
WebDDoS	528	0.001%
Total	48 269 665	100.0%

Table shows the class, the instances associated to it and the ratio according to the total amount of examples

3.1.14 CSE CIC IDS 2018

As the cyber-security datasets become focused on specific scenarios, there is also a need to represent company networks in a closer way. CSE CIC IDS 2018 [39] dataset was designed on such a structure with hundreds of computers representing the daily organization operation. With 1 simulated server room, 1 IT department, and 4 organizational areas, an attack network of 50 virtual machines targets 450 hosts. The structure provides Linux and Windows machines under 7 different attacks for 10 days. The final downloadable dataset²² provides the files for each of the days already labeled after the CICFlowMeter [41] extracted the 85 features. Researchers also made it available through Amazon Web Services²³. As it happened with the CIC IDS 2017 [10] dataset, the version used to extract the attributes had an error that would falsely tag some connections in a certain attack, or produce incorrect examples of normal flows [52]. The alternative downloadable version²⁴ includes 62 999 332 instances with 85 attributes distributed on 5 categorical, 1 timestamp, 78 numeric, and one multi-class label. In this paper, we used the modified version produced by Liu et al. [52]. Table 15 provides the main details regarding the class distribution in this dataset.

As a supporting tool for researchers, we provide Table 16 which presents a summary of the datasets that show the same

²² <https://www.unb.ca/cic/datasets/ids-2018.html>

²³ <https://registry.opendata.aws/cse-cic-ids2018/>

²⁴ <https://intrusion-detection.distrinet-research.be/CNS2022/index.html>

Table 15 CSE CIC IDS 2018 Distribution of instances according to the label

Class	Instances	Proportion
BENIGN	59 463 958	94.3882%
Botnet Ares	142 921	0.2269%
DDoS-HOIC	1 082 291	1.7179%
DDoS-LOIC-HTTP	289 302	0.4592%
DDoS-LOIC-UDP	2 527	0.0040%
DoS GoldenEye	22 559	0.0358%
DoS Hulk	1 803 160	2.8622%
DoS Slowloris	8 490	0.0135%
Infiltration - Communication Victim Attacker	203	0.0003%
Infiltration - Dropbox Download	85	0.0001%
Infiltration - NMAP Portscan	89 356	0.1418%
SSH-BruteForce	94 197	0.1495%
Web Attack - Brute Force	131	0.0002%
Web Attack - SQL	39	0.0001%
Total	62 999 332	100.0%

Table shows the class, the instances associated to it and the ratio according to the total amount of examples

attribute subset. The intersections show the standard used in the files with feature sets: CIC-Flow(CF) and NetFlow(NF). An empty intersection means the two datasets do not use the same standard. This allows a faster development of experiments needing less reconfiguration of methods.

3.2 Larger datasets

During this study, we found other datasets that are worth mentioning due to their relevance to some scenarios that researchers might find useful. Iot23 [60] and UGR [42] are large benchmarks oriented to the IoT and Real network captures respectively. Additionally, UQ NIDS v2 [33] joined datasets converted to the Netflow standard. The following subsections describe them.

3.2.1 IoT 23

The IoT poses challenges such as the amount of communication done with servers and devices. Different from desktop or laptop computers in which traffic is designed to be completed in one session, small technological equipment generates many fast complete connections. For the network flow dataset this grows as the number of considered devices grows. The IoT-23 [60] is a benchmark generated to identify malware in this context. It includes 23 files with 20 malicious scenarios and 3 benign ones. Downloadable files²⁵ show 9 different types of attacks. A total of 302 661 683 flows are in the dataset

²⁵ <https://www.stratosphereips.org/datasets-iot23>

Table 16 Compatibility between datasets regarding CIC-Flow (CF) or NetFlow (NF) PCAP extracted feature set

Dataset	CSE CIC 2018	DDoS 2019	Bot IoT*	Litnet	USB IDS	ToN IoT*	ISCX-2012	CIDDS	CIC IDS 2017	UNSW-NB15*	Hikari	NDSec-1	InSDN	OPCUA
OPCUA														–
InSDN													–	
NDSec-1	CF	CF							CF			–		
Hikari	CF	CF							CF		–			
UNSW-NB15*			NF			NF				–				
CIC IDS 2017	CF	CF							–					
CIDDS								–						
ISCX-2012							–							
ToN IoT*			NF			–								
USB IDS					–									
Litnet				–										
Bot IoT*			–											
DDoS 2019	CF	–												
CSE CIC 2018	–													

Intersections show the same standard used on both datasets

with 16 features distributed on 6 categorical, 9 numerical, and 1 multi-class label. It is important to highlight, that from all the datasets described in this paper, this is the biggest and the one that needs more work in terms of preprocessing.

3.2.2 UGR 16

Although real traffic is a recurrent mentioned characteristic on datasets, Maciá-Fernández et al. [42] consider that is challenging to be certain about the labels on network flows coming from these scenarios. In this regard, they present a benchmark that was generated while being connected to an Internet Service Provider (ISP) core network. The objective of the study was to provide a real-traffic dataset on top of analyzing the traffic cyclostationarity. As for the attacks, they were generated by controlled computers. Nevertheless, this does not assure that threats coming from the internet were not attempted. By being a part of the ISP infrastructure they could generate a dataset with normal, known attacks and background/uncertain real traffic. It is worth mentioning that known attacks contain not only the threats done by their controlled computers but the ones that anomaly detectors could identify on the background traffic [42]. The total set of flows is over 16,900 million captured in more than 4 months on NetFlow [47] Standard. Downloadable CSV files²⁶ referred to as calibration set are separated by each week for a total set of 17 files. A final 6-week testing set completes the benchmark for more than 200 gigabytes. This dataset includes 43 attributes distributed over 6 categorical, 1 timestamp, 4 numerical, and a multi-class label.

²⁶ <https://nesg.ugr.es/nesg-ugr16/>

3.2.3 NF UQ NIDS v2

By having a common feature set on datasets, Sarhan et al. [33] were able to merge UNSW-NB15, ToN IoT, BoT IoT, and CSE CIC 2018 into one final benchmark with 75 987 976 records. It is important to highlight that the final distribution is 66.88% for attack network flows and 33.12% for benign records. Although this joined scenario stands as a complete example, it also inherits characteristics from its parts. There are only 164 Worms threats or 1423 Shellcode samples and some of the attacks had to be renamed for them to be compatible [33]. While we consider this dataset to be an example of what can be achieved by using standards, it does not solve other problems such as imbalance. The file is downloadable²⁷ and includes 43 features distributed over 6 categorical, 37 numerical, one binary attack tag, and a multi-class label.

3.2.4 Alternative datasets

Each year new datasets are produced highlighting different scenarios and threats. Researchers and practitioners can download them for new algorithms and tests. In 2023, H23Q was released showcasing 10 diverse attacks against HTTP and QUIC services [61]. The set can be downloaded on the Aegean University site²⁸.

University of New Brunswick is a major contributor to NIDS datasets. On 2024, CICIoV2024 was released focusing

²⁷ <https://www.cyber.uq.edu.au/project/machine-learning-based-nids-datasets>

²⁸ <https://icsdweb.aegean.gr/awid/other-datasets/H23Q>

on DoS and spoofing attacks [62]. More than 30 datasets are available on the site, downloadable for free²⁹.

UNSW is also a major contributor to research, releasing NGIDS-DS on 2023 claiming to be a realistic Intrusion Detection Systems dataset [63]. Is downloadable for free on the UNSW site³⁰.

Stratosphere Lab provides the well-know IOT-23 [60] as well as CTU-13 [64] datasets. The lab focuses on bot detection on IoT networks and has released 6 datasets on malware recorded on real scenarios. The datasets can be downloaded on the lab site³¹.

There are other less popular datasets such as UNICAUCA [65] and VHS-22 [66] datasets containing millions of network flows and downloadable on Kaggle^{32 33}. Nevertheless, more study on both is still missing.

3.3 Dataset preprocessing

Network Cyber-security applications are deployed in a variety of environments ranging from two devices up to millions of them. Then, for any method to be tested, there is a necessity for scenarios that display the challenges and opportunities of methods. Traffic characteristics, speed, and threats have different behaviors, expectations, and handling. Researchers produce datasets to display these alternative configurations and showcase the most adequate methods [31, 37, 56].

A challenging issue with the different benchmarks is the feature extraction process. The applications done for the feature extraction process have different philosophies and export diverse perspectives of the data. While the standard Netflow [47] in its version 9 is propagated, programs such as CICFlow Meter [41], Argus[45] and Zeek/Bro [46, 57] are available and used. The amount of attributes exported from this stage has been reported as low as 10 [34] and as large as 85 [10, 39], or even more on customized deployments. In the same way, the type of attributes can be categorical and numerical. Some researchers, such as Sarhan et al. [33] have provided alternative versions of the same PCAP files of a benchmark, so new research can benefit from the standardized format. It is important to highlight that each feature extraction process has different rules for connection terminations and timeouts which yields different flows between solutions.

Surveys identify some of the available datasets for network flows, but every day researchers are producing updated or new versions of them. Different and recent attacks are responsible for the contexts and environments for IDS, and the testbeds need to be updated as well. While methods still test their performance on old and outdated datasets [67] such as NSL-KDD [14] and KDD99 [15], options are available for IoT, corporate, and metropolitan network scenarios. Researchers are trying to develop benchmarks on realistic ways or even working infrastructures, but there is still a heterogeneous environment that methods should adapt to.

Two main challenges exist with datasets: the formats and the size. Cybersecurity networks can be studied in packet or flow-based records, and different types of files are designed for those. For the former challenge formats under unsupervised learning, registers are adequate once they are captured. If a model is trying to develop supervised learning, then these records should be labeled and exported to another format such as CSV. For the latter challenge, aggregation and optionally labeling steps must be performed. The final file is commonly a CSV as well. From the PCAP original files to the final ones, the size could grow from tens or hundreds of megabytes to tens of gigabytes. This process is shown in Fig. 2. It is common to find the datasets in a compact and separated way to make them available for different users. Even in this readable version, they can include duplicated, invalid, and null values.

For this paper, we have downloaded each of the datasets from the original site, commonly a research web page from a university. As the search for benchmarks advanced, alternate versions of CIC IDS 2017 [10], UNSW-NB15 [11], ToN Iot [56] or Bot IoT [58] among others appeared, some of them having great differences regarding the original one. It is up to the cybersecurity researcher to understand and decide which specific file to use, as it has been shown that performance differs from one to another [33]. The details of each dataset regarding technical characteristics and download links can be found in Section 3.1.

3.3.1 Dataset challenges

Certain datasets need a specific treatment. We highlight the ones that we consider researchers should look closer to as they are standard but do present some challenges to obtain and process.

CIC IDS 2017 [10], is a dataset part of the New Brunswick Canadian Institute for Cybersecurity. It is accessible after an automated form to finally download 5 files with their labels. A folder with PCAP files is also available on the site. Although is easy to compile, it has been reported that it contains errors due to CICFlow Meter [41] extractor. Should researchers want to avoid a PCAP extraction process, a complete file

²⁹ <https://www.unb.ca/cic/datasets/index.html>

³⁰ <https://unsworks.unsw.edu.au/entities/dataset/fa64cd6b-4a80-49bb-80fe-23549a60d695>

³¹ <https://www.stratosphereips.org/datasets-overview>

³² <https://www.kaggle.com/datasets/jsrojas/labeled-network-traffic-flows-114-applications>

³³ <https://www.kaggle.com/datasets/h2020simargl/vhs-22-network-traffic-dataset>

download is provided by Engelen et al. [16] site³⁴. Once the file is downloaded, network flows with “Attempted” label can be safely converted to Benign/Normal traffic.

CSE CIC IDS 2018 [39], available only through the Amazon Web Services Common Language Interface (AWS CLI), contains gigabytes of network flow information. As it has been reported, this dataset also contains errors due to CIC Flow Meter [41]. A complete, lighter version is available on the site³⁵ developed by Engelen et al. [16]. Once the file is downloaded, network flows with the “Attempted” label can be safely converted to Benign/Normal traffic.

DDoS 2019[38], available as a download on the New Brunswick Canadian Institute for Cybersecurity, is unclear if it is affected by the CICFlow error [41]. As the PCAP files are available, we used a modified version of CICFlow to extract the features and then a custom script to label them according to time and IP.

CIDDS [34], is a dataset from Coburg University with two scenarios. These are compatible and can be concatenated to generate one final benchmark. The 6 downloadable files (4 of CIDDS-1 and 2 of CIDDS-2) have the same columns but cleaning must be performed on each of them as there are invalid ports. For this paper, we erased the invalid records.

NDSec-1 [35] of the University of Applied Sciences Fulda, needs 4 files, all from a direct download on its site. These can be joined directly as they all share the columns.

USB IDS [37] is a dataset of the University of Sannio. The published version is already divided into Training, Validation, and Test CSVs. In this paper, we joined the files to pre-process the records and generate our own sampling.

On the other hand, OPCUA [31], InSDN [32], Hikari [17], Netflow version of UNSW-NB15 [33], Netflow version of Bot Iot [33], Netflow version of Bot Iot [33], Netflow version of ToN Iot [33] and Litnet [12] are available on a direct download link with one zipped file ready for processes.

Once all the datasets were downloaded and joined, as in the case of the multiple files, pre-processing was applied. For each of them, a pandas version was issued, identifying the specific type of data (float, integer, or string) and data size (in bytes) to optimize the file size. A pickle dataset was saved for faster loading on the cluster. These steps are summarized in Fig. 1.

On the pickle version benchmarks, ID and timestamp columns were dropped as these are not useful in classification. As a final step, stratified sampling was performed to separate training (70%), validation (15%), and testing (15%). Each method had code to either take these datasets as separate files or mix training and validation while maintaining

testing differentiated. The final dataset size in pickle format is shown in Table 17. All preprocessed datasets are provided in a pickle format at <https://github.com/UOttawa-Cyber-Range-Research/GuardiansOfTheNetwork>.

3.4 Methods

Different ML techniques have been applied to intrusion detection. Decision Trees and their variations are deployed in fields such as flight delay prediction, financial forecasting, or sentiment analysis [68]. Their non-linear capability as well as categorical dimension handling is desirable for networks, having top scores on [25, 69–71]. Recently, Yang et al. [8] developed a system capable of identifying different types of attacks on settings such as cars. LCCDE [8] method is an ensemble of three classifiers, namely LightGBM, XGBoost, and CatBoost. Decision trees based on Boosting are a technique that implies the generation of a series of weak learners/models, weighted by their ability to detect certain classes. On each training iteration, a new decision tree is generated with attention to the errors of the previous one [8]. The final model is the collection of the generated trees with an aggregation operation capable of deciding a class of new observations. In the cited technique, three gradient-boosting classifiers are trained separately on the original dataset. During classification, a set of rules is applied to decide the learner that should be regarded as final for the specific observation. The technique reported was tested on a common benchmark, the CIC IDS 2017 dataset [10]. Being compared to techniques such as K-Nearest Neighbors, Random Forest, and Deep Belief Networks, it shows an F1 Score of 99.811% on their tests.

Table 17 Dataset sizes in MB

Dataset	Size in MB
OPCUA	11
InSDN	79
NDSec-1	139
Hikari	169
UNSW-NB15*	310
CIC IDS 2017	566
CIDDS	688
ISCX-2012	1 036
ToN IoT*	2 184
USB IDS	2 307
Litnet	4 471
Bot IoT*	4 852
DDoS 2019	14 196
CSE CIC 2018	17 789

* Using variations from [33]

³⁴ https://intrusion-detection.distrinet-research.be/CNS2022/Dataset_Download.html

³⁵ https://intrusion-detection.distrinet-research.be/CNS2022/Dataset_Download.html

Although researchers have been implementing techniques with SVM (Support Vector Machines), Naive Bayes, or Random Forests [19], new inspirations from other fields have found their way with competitive results. Pontes et al. [72] developed a model based on the Inverse Potts Model, originally explored in Quantum Mechanics. Firstly implemented for anomaly detection, the technique is claimed as an explainable ML method different from Neural Network models regarded as black-box procedures. By inferring a statistical distribution based on the training sample, the model gets its inspiration from a “mathematical description of interacting spins on a crystal lattice” [72]. Each of the features in the dataset is regarded as a node with associations to every other attribute, connected with edges that are weighted with a coupling value. The combination of discretized feature values estimate the total energy of the system which is identified as a known (low energy) configuration or as an unknown (high energy) one. This definition was later extended [73] to detect not only benign and abnormal traffic, but also to implement multi-class output. Internally, the new model would generate a series of submodels, one for each label included on the dataset. The final assigned class to a new observation is the one with the least energy, provided it is under a threshold. Should this not be the case, the procedure can determine that no class should be assigned, therefore also being able to detect zero-day attacks. Experiments done on a CIC IDS 2017 [10] dataset version show a Macro F1 score of 75.2%. Although this technique presents a broad opportunity for enhancement, it is remarkable that no class balancing technique was applied. The results show that three highly imbalanced classes lower the metric; the authors explained that other tags with a higher number of examples display a state-of-the-art performance. A second experiment was implemented on a different dataset version, highlighting unknown attacks after synthetically removing class assignments. Results outperform OCN [74] and ODIN [75] techniques by 23% and %6 respectively, previously regarded as state-of-the-art methods.

High performance ML techniques have developed recently in fields such as anomaly detection. Deep Learning, an area that uses Perceptrons (artificial neurons) arranged in layers, has shown top results for problems in image analysis [76], language processing [77], and even generative artificial intelligence [78]. NIDS have been developed with these techniques. As Anomaly Detectors, Neural Networks (NN) have been proven effective [27] with accuracy ratios of up to 99.99%. For the multi-class settings on network intrusion detection, efforts are still being made to enhance metrics while accounting for dataset imbalance [79].

A basic model in Deep Learning (DL) is the Deep Neural Network with Fully Connected Layers. This method implies an input layer for each of the attributes and an output layer with at least one neuron per label for multi-class scenarios

[4]. Any number of hidden layers can be set in between, connecting to the next one with a set of weights for each of the edges. The training of the model is done by iteratively applying the backpropagation algorithm and changing the weights after estimating the value of an error function between the desired output and the actual output. Different architectures can be trained according to desired metrics and specific behaviors [22]. Basnet et al. [80] developed an architecture for an IDS with 1 input, a 128-neuron hidden layer, and 1 output layer. The study is oriented towards measuring the performance of this model with different technologies; nevertheless, confusion matrices reported for CSE-CIC-IDS 2018 [39] display high True Positive values for the majority of the classes. However, it shows two classes without examples on the test set, which does not allow to fully understand the behavior of the model.

One of the DL neural network variations is the Restricted Boltzmann Machines (RBM) [27]. This unsupervised learning architecture is built with a visible input layer and just one hidden layer. As the latter is not considered the output, it can have any size that the designer decides. To train, RBM uses a Contrastive Divergence equation instead of Cross Entropy Loss, common on multi-class DL models [21]. This allows the model to measure the influence of joint configurations in the visible and hidden layers, and eventually apply small modifications to the weights. Belarbi et al. [81] implemented an array of stacked RBM, known as a Deep Belief Network (DBN). For this architecture, each hidden layer of an RBM is the input layer of the next one and a final Fully Connected Layer produces the classification. Due to the combination of architectures, the authors state that two stages are needed: first, the method pre-trains the RBMs to reconstruct the inputs; second, it finishes the calibration of the weight through fine-tuning for the RBM and the Fully Connected Layer neurons.

This method reports a macro precision of 88% and a macro recall of 99% for the CIC IDS 2017 dataset [10]. Nevertheless, the total amount of classes calculated is 6, instead of the original 16. On top of this modification, the study also applied SMOTE [82] on the minority classes to balance the examples.

Convolutional Neural Networks (CNN), another DL architecture is the basis for image analysis and recognition, although it has been applied in 1-dimensional and time series datasets as well [18]. By applying certain operations on spatially close features, it can leverage relationships and patterns [27]. This type of network applies two main operations: convolution and pooling. While these two are responsible for the feature extraction process, the former is applied to generate representations of the data while the latter selects values based on a neighborhood. CNNs last layer is commonly a Fully Connected Layer after a flattening operation that receives the convolution extracted attributes and clas-

sifies the observation. The training process works in the same way as regular Neural Networks also modifying the weights of the filters applied in the convolutional layers [21]. The neighborhood in which a CNN works is defined by the network designer. In this regard, it is possible to develop an N-dimensional component; nevertheless, 2-dimensional filters are the most common. To work with tabular data, where each row is considered independent of another, there is only one dimension; the row itself. Attempts to generate two-dimensional data out of rows have also been developed though they will not be discussed in this paper. In this way, 1-dimensional convolutional operations allow CNNs to be applied for cybersecurity network flows applications. By considering the filters as feature extractors, Akgun et al. [59] designed a method for multi-class classification and tested it on CIC IDS 2019 [38]. By implementing four different DL models, their proposal based on three convolutional steps scored 97.3% on Macro F1 score. As the authors reported, steps were performed on the dataset, just before the technique could be applied. A reduction of 98.7% from the original dataset was done after duplicate removal, feature selection, and undersampling procedures. Although the paper reports state-of-the-art results, it is unclear if the report is the Macro F1 score or accuracy, which is labeled on the report. On top of this, no previous method was implemented on the same version of the dataset.

Aside from the DL basic architecture, a component that can be modified according to specific requirements is the Loss Function. As this is the final part of the training Forward stage and the first of the backpropagation, it gets an important role in new method design. Supervised Contrastive Loss (SCL), proposed by Khosla et al. [83] reports better results for multi-class classification on fields such as Computer Vision [84] and Natural Language Processing [85]. This recent technique uses augmented samples produced from a set of training ones [83]. It is very important that these two variations have different representations but keep the original labels. A way of performing this step is through the use of DL Encoders, which map a set of features to a different space, likely with fewer attributes [86]. By only training two encoders, without the decoders, different representations of one specific observation are generated. These two examples will be used by the Contrastive Loss Function that is used instead of the Cross-Entropy Loss and allows the backpropagation method to work [83]. SCL authors propose a specific architecture using all these concepts and claim that the technique will train the network in such a way that it will separate positive samples and negative ones with a larger difference on each iteration. Conflow (CFC) [87], is a recent method for cybersecurity network flows classification that uses Contrastive Loss and Cross Entropy Loss in a weighted scheme. On top of this, they propose adding two architectural blocks named Dense Resnet. This model proposed by

Huang et al. [88], introduces a main advantage to DL as it is a countermeasure for the Gradient Degradation problem [89]. By having two networks with the same input and training at the same time with common loss functions, they achieve the generation of synthetic samples and state-of-the-art results. Authors tested the method on ISCX-2012 [36] and CIC IDS 2017 [10] extracting the features directly with the PCAP Software NFStream [90]. State-of-the-art methods were outperformed with Macro F1 scores of 99.16% and 99.60% in complete versions of both datasets.

Networks, due to their inherent characteristics can be represented as a set of nodes connected by edges. This structural perspective inspires a DL technique known as Graph Neural Networks (GNNs) [91]. The objective of the method is defined according to the requirements as it can be either for node tagging, complete graph categorization, or edge classification. This last case is specifically what researchers on cybersecurity network flows datasets are looking for, being able to determine if a certain communication between two computers can be deemed as normal or part of an attack. GNNs generate an embedding representation from the nodes and edges maintaining the original connectivity, but there are several ways to achieve this. Lo et al. [9] based their proposal on a specific procedure known as E-GraphSAGE [92] (EGS). This method makes use of depth layers and aggregator functions to develop a view of the nodes and their neighborhood. Considering that the differentiable aggregator functions capture the adjacent nodes' relation with the current one, they can output a final embedding vector that will be classified by a classical Neural Network such as a Multi-Layer Perceptron. Authors claim that an important difference between the original GraphSAGE [92] algorithm and theirs, is that their method integrates edge features on sampling and aggregations. As nodes attain no features on this task, they are just regarded as one-vectors. Experiments on this method show different results for the same network structure with alternative features. Bot IoT [58] and ToN Iot [56] are two datasets developed by the University of Canberra. Their original versions include 44 features and identify the nodes from which the attacks were produced. Sarhan et al. [33] used the same PCAP files but presented them with Netflow standard producing two alternate versions: NF-Bot-Iot and NF-Ton-Iot. Lo et al. [9] tested their method on both versions of the two datasets. Although both maintained the same structure and labeling procedure, data varies in terms of records and class proportions and it may be regarded as two different problems. Nevertheless, Macro F1 scores results were compared on original versions and Netflow versions. The performance on original dataset versions reached 100% on Bot IoT [33] and 87% on ToN Iot [33] while it was worse on Netflow versions with 81% and 63% respectively. Authors provided a comparison with KNN (K-Nearest Neighbors), XGboost, and Extra Tree classifiers [9]. The method displays the same

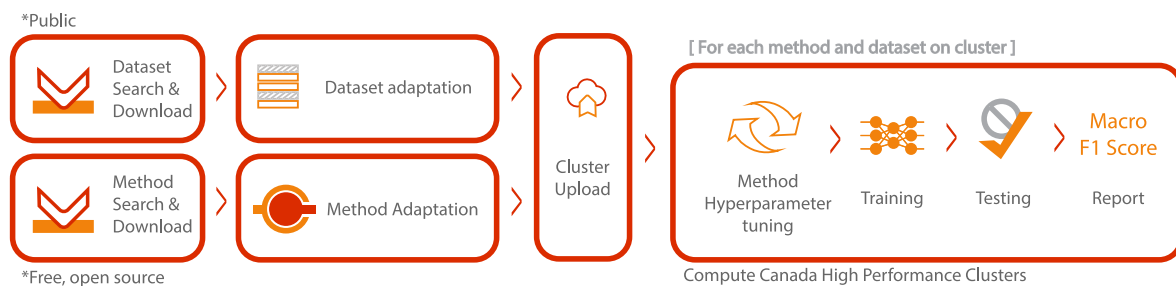


Fig. 3 Experiment flow diagram: We obtained the public datasets and free, open source algorithms, adapted them for cluster operation, performed the hyper-parameter tuning and reported the final Macro F1 Scores for comparison

metric value for the ToN Iot dataset and a 3% improvement for the rest. Is remarkable that this technique is able to be visualized maintaining an explainability degree that is difficult to obtain in other DL algorithms.

Surveys in cybersecurity [3, 4, 18, 23, 93] allow researchers to start with an overview of the techniques and datasets that other studies have been involved with. Nevertheless, just from a few surveys one can reproduce some of the reported results. While we appreciate this information, there is still a gap between the proposed methods and the obtainable yet comparable benchmarks and techniques. This paper focuses on not only reporting recent results but also on showing a working perspective of the cybersecurity field. In the next section, we describe a protocol that researchers should follow to get, deploy, and test solutions on different datasets. Our main objective is that researchers can plan better the steps needed for their own proposals and start the development in a faster way.

3.5 Experimental methodology

Proposals, especially in the Deep Learning field have been majorly implemented on cybersecurity datasets. Aldweesh et al. [27] report 11 from 2018, Khan Adawadkar and Kulkarni [28] report 2 between 2019 and 2021, Macas et al. [3] report 11 between 2019 and 2021, and He et al. [94] report 5 more between 2019 and 2020. Among cybersecurity proposals, there are two main types of problems: anomaly detection and attack classification. While the former has standard metrics and protocols, the latter presents gaps when trying to compare different studies. Recent studies [9, 13, 95, 96] have been able to display their solution working on 1 and up to 5 benchmarks, most of them being NSL-KDD [14], KDD99 [15], CIC IDS 2017 [10] or UNSW-NB15 [11] as reported by [21]. Although these are still the standard, other datasets showing updated scenarios and attacks have already been developed, and researchers are able to use them. A wider, up-to-date, and unified benchmark is needed to carry out the comparison of the different approaches.

In addition, some attempts to train models between datasets have also been carried out. XENIDS from Apruzzese et al. [13] studies a framework and a method implementation capable of interacting with different benchmarks. The final proposed solution had to select a common subset of attributes to finally work, and it offered better results when samples from one dataset were borrowed to the other. As this is not strictly transfer learning [97], only shows a possible research direction, yet to be investigated.

By taking advantage of the new research benefits from the shared attempts of colleagues, we have searched for publicly available network flow based methods with different techniques that could provide baselines for upcoming studies as shown on Fig. 3. The final 7 selected methods, which were discussed in the previous sections, are as follows: Leader and Class Confidence Decision Ensemble³⁶ (LCCDE) [8], Energy Flow Based Classifier³⁷ (EFC) [73], Fully Connected Neural Network Model³⁸ (NN) [80], Convolutional Neural Network Based Model³⁹ (CNN) [59], Deep Belief Network Model⁴⁰ (DBN) [81], E-GraphSage Based Classifier⁴¹ (EGS) [9] and Contrast Network Flow Classifier⁴² (CFC) [87]. All of them have their code available on GitHub and contain instructions to run the code on at least one dataset.

Due to the different objectives of the mentioned methods, some of these methods are coded in a Jupyter Notebook, instead of a regular Python file. Although not technically challenging, this introduces an extra step that had to be planned. We made modifications to general interface mechanisms on almost all of them to accomplish a standardized version: a) shell call as an argument to Python compiler b)

³⁶ <https://github.com/Western-OC2-Lab/Intrusion-Detection-System-Using-Machine-Learning>

³⁷ <https://github.com/EnergyBasedFlowClassifier/EFC-package>

³⁸ <https://github.com/Colorado-Mesa-University-Cybersecurity/DeepLearning-IDS>

³⁹ https://github.com/dwday/deep_learn_ids

⁴⁰ <https://github.com/othmbela/dbn-based-nids>

⁴¹ <https://github.com/waimorris/E-GraphSAGE>

⁴² <https://github.com/AshinWang/ConFlow>

sets of comma-separated column names to be erased from the input dataset c) sets of comma-separated column names to be regarded as categorical attributes d) batch and epoch size where applicable and e) three datasets naming convention regarding train, validation and testing. This way, all the methods were able to be executed in a similar way by shell scripts to be run on Compute Canada High-Performance Cluster. A summary of these steps can be found in Fig. 4.

After finding and obtaining the code for each algorithm, we focused on generating a common shell call for all of them. Some of the methods such as CFC [87] and DBN [81] were developed having an advanced argument parser layer and configuration files, respectively. Others such as LCCDE [8] and EGS [9] were developed as Jupyter notebooks, which implied an extra step to make them stand-alone. Either way, code that was too adapted to original test datasets had to be removed or modified as this was oriented towards experiment repeatability more than new scenarios tests.

In this paper, the datasets were fed to the methods as complete as possible, since previous works reported that the different techniques tailored the datasets to their needs without considering every feature. EGS [9] kept Source and Destination addresses while being erased for the rest. NN [80], CNN [59], and DBN [81] accepted only numerical values, so categorical ones were dropped at run-time. EFC [73], LCCDE [8], and CFC [87] worked with categorical data so only Source and Destination addresses and ports were dropped.

The output was also standardized. Confusion matrices and classification reports were obtained throughout the different implementations. Each time the algorithm was run on a certain dataset, the reports would be recorded on a file and stored for later analysis. The code modification was straightforward as the majority of the methods included standard naming conventions on their final predictions.

To summarize, codes were adapted to have a common shell input interface and a standard confusion matrix output. In this process, we modified any reference for specific datasets focusing on the structure done on the deployment cluster.

3.6 Metrics

Methods training with Cyber-security Network Flows datasets must be measured with metrics that are resistant to large class imbalance. Research [9, 18, 19, 69, 72, 93, 98] suggests the usage of the F1 score. For the multi-class case, the macro-averaged version is recommended.

For a given class C :

- TP: instances classified correctly as being of class C
- TN: instances classified correctly as not being of class C
- FP: instances classified incorrectly as being of class C
- FN: instances classified incorrectly as not being of class C

The F1 score is the harmonic mean of Precision and Recall, as shown in (3) and provides a global view of performance [99]. The macro average gathers the results of the different class cases and averages them. As opposed to accuracy it is not biased towards the majority classes. For this reason, we selected this metric for the performance assessment of our experiments.

$$F_1 = \frac{2 * P * R}{P + R} = 2 * \frac{\frac{TP}{TP+FP} * \frac{TP}{TP+FN}}{\frac{TP}{TP+FP} + \frac{TP}{TP+FN}} \quad (3)$$

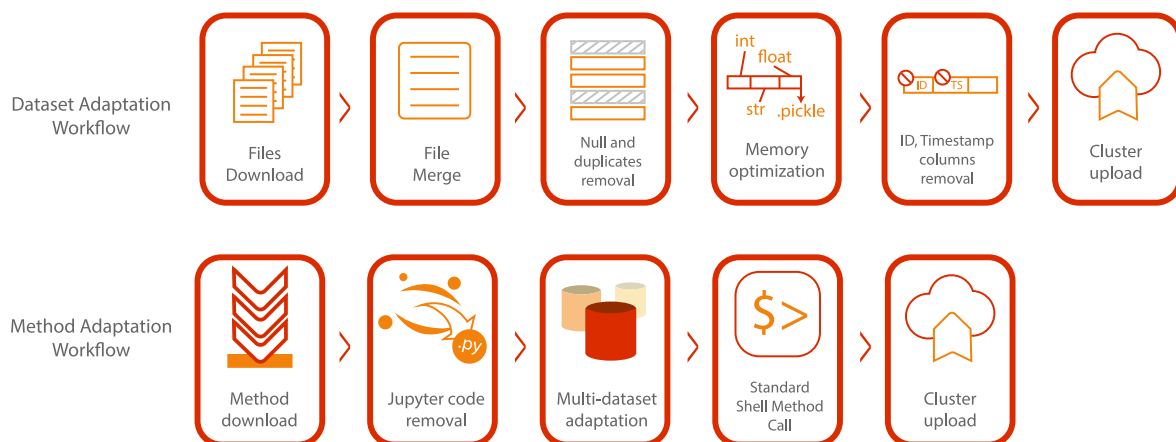


Fig. 4 Steps needed to adapt algorithms and datasets to a common interface that allowed uniform testing

4 Experimental evaluation

4.1 Experimental settings

In this study, 7 methods were compared on 14 network datasets. The benchmarks originate from PCAP files generated by network device logs. Applications such as CIC Flow Meter [41] or Netflow [47] export an aggregated version of the communications between two computers at a given time, known as a flow. After a set has been exported, it is labeled in function of IP Addresses and Timestamps in accordance with a previously defined schedule, as shown in Fig. 2. If the dataset was not obtained in a lab setting, analysis tools tag the threat by a set of rules. Cybersecurity solutions analyze the dataset and classify the flows with a benign or an attack specific category. The techniques challenged included shallow ML and Deep Learning techniques on benchmarks ranging from 107 634 instances to 48 269 665 samples. All the methods were implemented on Compute Canada High-Performance Clusters with an execution limit of 7 days on one 32-processor node. Details on the specific CPUs (Central Processing Units) can be found on Compute Canada Cedar Cluster Wiki⁴³. GPU (Graphics Processing Unit) was enabled for methods developed with explicit code designed for this option.

Every dataset was pre-processed to avoid the inclusion of ID and timestamp attributes. The methods were modified to drop any other feature that was not considered, such as categorical values for DBN [81], NN [80] and CNN [59]. As it was analyzed in the code, some methods included the source and destination port attributes in the feature set. We do not consider that port is neither a numerical feature nor that it should be regarded as a possible value between 0 and 65535. For this reason, we added 2 categorical attributes to every dataset. The source and destination port are a category that is a number between 1 and 3 that identifies them as well-known, dynamic, and ephemeral [29].

The same splits were used as input to each algorithm having a common training set (70%), validation set (15%), and testing set (15%). If the procedure did not need the validation set, it was merged with the training set.

We considered a hyperparameter grid search if the dataset was not part of the benchmarks for the method. Neural networks were tested with different epochs and batch sizes, nevertheless, the objective was not to determine the best procedure but to provide a general characterization of it. On top of this, each training set was provided in the most complete way, without feature engineering and only dropping attributes that could not be considered. For this reason, NN [80], CNN [59], DBN [81] removed categorical features while EFC [73], LCCDE [8], EGS [9] and CFC [87]

kept them. While we acknowledge that a more thorough study of each method and dataset pair regarding feature engineering can output better results, we present the results as a way to provide a wide perspective of their characteristics and opportunities. Table 18 provides a summary of the technique and hyper-parameters each method provides. All the code for the algorithms used in our experiments is available at <https://github.com/UOttawa-Cyber-Range-Research/GuardiansOfTheNetwork>.

Macro F1 score was chosen as the performance metric for its properties on multi-class imbalanced datasets and easier comparison to other studies. Other measures such as accuracy are not suitable due to the major imbalance in the datasets. In this regard, 5 out of the 7 datasets shared this metric and reported by [4, 18, 94]. The average that provides between precision and recall allows a unified metric. Averaging the results provides a final compound estimation on the dataset.

4.2 Results

In this study, we used 14 Network flow datasets, applied 7 different methods, and measured the corresponding Macro F1 scores. The results show different performance clusters regarding each of the techniques applied. We also obtained the ranks by dataset and averaged them to detect performance patterns on each method. While the objective was not to identify the best method on these benchmarks, the mean score shows CFC [87] as the top performance algorithm, followed by LCCDE [8] and NN [80]. This section presents the results analyzing each method in particular and a summary of the methods, their average rank, and average Macro F1 score is shown in Table 19.

To rank the algorithms, the Friedman test with Nemenyi post-hoc [100] test was applied setting 0 if a method could not complete a certain dataset. The Critical Difference Diagram is shown in Fig. 5 and displays the CFC [87] method as the best ranked method, although without statistically significant evidence when considering the LCCDE and the NN methods.

CFC [87] is a proposal that considers imbalance and uses Supervised Contrastive Learning with Dense Resnet Neural Networks. Achieving the top mean f1-score and the least standard deviation, it ranked 1.79 completing 13 out of 14 datasets. It is worth mentioning that CFC [87] had these scores while keeping the standard configuration and considering categorical features, making it a fast implementable method to test new proposals.

LCCDE [8] ranked 2.46, second on all tests. It could finish successfully 13 out of 14 with a memory problem on the Litnet [12] dataset. As a decision tree based method, it can accept categorical as well as numerical values with no tuning at all. The code had SMOTE [82] to balance datasets but, to keep all benchmarks equal to every technique, we modified the code to disable it.

⁴³ <https://docs.alliancecan.ca/wiki/Cedar>

Table 18 Hyper-parameters defined as modifiable by each of the methods and search space used for hyper-parameter tuning

Method	Technique	Hyper-parameters	Search Grid
LCCDE	Boosted Trees	None	
EFC	Inverse Potts Model	Cut-off quantile	Cut-off quantile: {0.4,0.6,0.8}
NN	Neural Networks	Epochs, Batch size	Epochs: {20,50,100} Batch Size: {10,100,500}
CNN	Conv. Neural Networks	Epochs, Batch Size, Learning Rate	Epochs: {20,50,100} Batch Size: {10,100,500}
DBN	Deep Belief Network	Data Loader Batch Size, Epochs, Optimizer Weight Decay, Loss Function, Per RBM unit: {Epochs, Batch size, Learning rate, Decay, Momentum, k}	RBM unit batch Size: {64,128} RBM unit epochs: {10,20}
EGS	Graph Neural Networks	None	None
CFC	Dense Resnet Blocks Neural Networks w/ SCL*	Loss Method, Loss Function Temperature, Epochs, Batch size, Loss Functions Weights, Learning Rate, Weight Decay, Dropout	None

*SCL: Supervised Contrastive Loss

If empirically, the method showed no major positive difference in classification, the hyper-parameter would be fixed to the value suggested by its authors

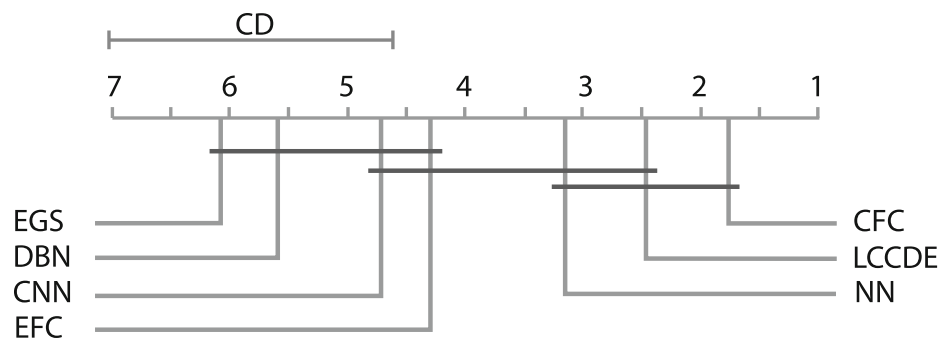
Table 19 Macro F1 Score results from each experiment

Datasets	Learning Methods							Avg
	LCCDE	NN	DBN	CNN	EGS	EFC	CFC	
OPCUA	1.000	1.000	0.200	0.850	0.720	0.870	0.980	0.803
InSDN	0.990	0.980	0.460	0.830	0.070	0.810	0.952	0.727
Hikari	0.610	0.590	0.210	0.630	0.060	0.480	0.600	0.454
UNSW-NB15*	0.630	0.460	0.100	0.330	0.25	DNF	0.660	0.405
CIC IDS 2017	0.960	0.970	0.240	0.010	0.500	0.860	1.000	0.649
CIDDS	0.800	0.250	0.250	DNF	DNF	0.540	0.990	0.566
NDSec-1	0.730	0.460	0.090	DNF	0.580	0.050	0.800	0.452
ISCX-2012	0.850	0.380	0.200	DNF	DNF	0.560	0.990	0.596
ToN IoT*	0.910	0.870	0.250	0.530	DNF	DNF	0.938	0.700
USB IDS	0.310	0.450	0.150	0.420	DNF	0.480	0.690	0.417
Litnet	DNF	0.120	0.070	0.130	DNF	0.170	DNF	0.123
BoT IoT*	0.950	0.900	0.570	0.820	DNF	DNF	0.990	0.846
DDoS 2019	0.440	0.390	DNF	0.040	DNF	0.330	0.590	0.358
CSE CIC 2018	0.910	0.840	DNF	DNF	DNF	0.880	0.990	0.905
Average	0.776	0.619	0.233	0.459	0.363	0.548	0.859	

* Using Netflow variations [33]

DNF: Did Not Finish. The results showing this message were not completed on the time constraints, failed due to memory consumption, or were not able to start due to dataset characteristics

Fig. 5 Critical Difference Diagram after Friedman test with Nemeyi Post-hoc ranking the 7 methods



NN [80], as a fully connected neural network and 1 hidden layer architecture, ranked 3.14 on average and could consume every dataset. It achieved 90% or more on BoT IoT [58] and CIC IDS 2017 [10] which is aligned with the rest of the methods that scored high on these. It required the least amount of time to finish, having less than 12 hours to complete the largest dataset with performance comparable to the rest.

EFC [73], a model based on the Inverse Potts Model ranked 4.29 with 11 out of 14 benchmarks done. This method considers categorical attributes as well as numerical ones and does not need a different configuration other than the default to train. From their examples, one hyper-parameter is trained. Three configurations were tested, being the default cutoff quantile equal to 0.8 the best in every case. Three benchmarks generated an error regarding memory management. This was not specific to the set of features, as other datasets had the same attributes and could be analyzed.

CNN [59] is a 1-dimensional convolutional that ranked 4.71 on average and presented some problems due to its dependency on a minimum amount of numerical features on the dataset. This prevents the method from working with some benchmarks such as CIDDS [34], NDSec-1 [35] and ISCX-2012 [36], hence only achieving analysis on 11 from 14. For the rest of the benchmarks, it fails to produce high f1-scores. As the method was originally tested on DDoS 2019 [38] with feature engineering and resampling processes, it depends heavily on balanced datasets and selected important features.

DBN [81], the deep-belief network ranked 5.57 on average and shows a poor performance but 12 out of 14 datasets. Although we consider this to be the result of coarse tuning, the method implies a high quantity of hyper-parameters including architectural changes, which were not thoroughly tried during tests due to time restrictions. On top of this, authors [81] modified the benchmarks to have more balanced datasets, which explains the different results on the same benchmark, CIC IDS 2017 [10].

EGS [9] is a graph neural network method that tries to identify the structure of both: the different samples and the complete network. It was the lowest ranked method and required the longest time to finish. The results show that it could complete less than half of the benchmarks due to its resource consumption. This method considers categorical and numerical data and was tested without modifying any hyperparameter.

5 Discussion

NIDS development has grown with the digital transformation and communications landscape. Only in the last 4 years, [101–103] report more than 25 proposals, targeting ML methods focused on intrusion detection. While this high-

lights the importance of NIDS, [103] report that only 4 datasets represent 72% of the test cases. KDD99 [15] and NSL-KDD [14] majorly represent half of these datasets to be tested with new methods but include attacks and scenarios already outdated [67]. New proposals and solutions must account for contexts such as IoT, mobile and metropolitan networks such as datasets made by [12, 56, 58, 104].

Our study of NIDS states 14 recent datasets and tests 7 ML methods, all of them open and freely available. Our goal is to introduce new practitioners and research teams to updated use cases and present benchmark methods showcasing recent techniques. To better understand the solutions and the updated Network Flows datasets we tested them and discuss their key characteristics. This section presents the main findings on Subsection 5.1 for the methods and datasets, identifies the solutions' implementation implications on Section 5.2, compares other studies in Section 5.3, discusses the implications for new practitioners and researchers on Section 5.4 and states limitations along with general recommendations in Section 5.5.

5.1 Main findings

Generalizing and contrasting the methods' performance is challenging due to hyperparameter tuning, feature extraction, and dataset pre-processing. On top of this, older and outdated datasets such as NSL-KDD [14] and KDD99 [15] are still being reported on surveys. In this paper, none of the methods used these old benchmarks. Instead, we focused on testing the developed methods on the existing newer datasets. As far as we know, this execution is the largest study in terms of methods and datasets used, showing properties and characteristics that allow practitioners to have a wider perspective of the solutions available. Figure 6 shows the results from the different methods grouped by dataset.

Methods Performance CFC [87] scored best on 10 out of 13 completed cases. While one of its main characteristics is the resistance to highly imbalanced datasets, its performance improves as the instance count grows. OPCUA [31], InSDN [32] and Hikari [17] datasets have less than 600,000 observations and rendered LCCDE [8], NN [80] and CNN [59] methods with the same or better performance than CFC. Once the datasets grow, as Fig. 7 shows, this method displays a clear advantage, even with a high imbalance, provided it could finish the training. Figure 8 shows the whisker plot for the methods and displays the highest scoring average and minimum variation of the algorithms tested, followed closely only by LCCDE [8]. CFC [87] had a Macro F1 score of 100% for [87]; this is consistent with LCCDE [8] performance and NN [80] which had a 96% and 97% respectively, and the metric reported on the original work [87] of 99.96%. [18].

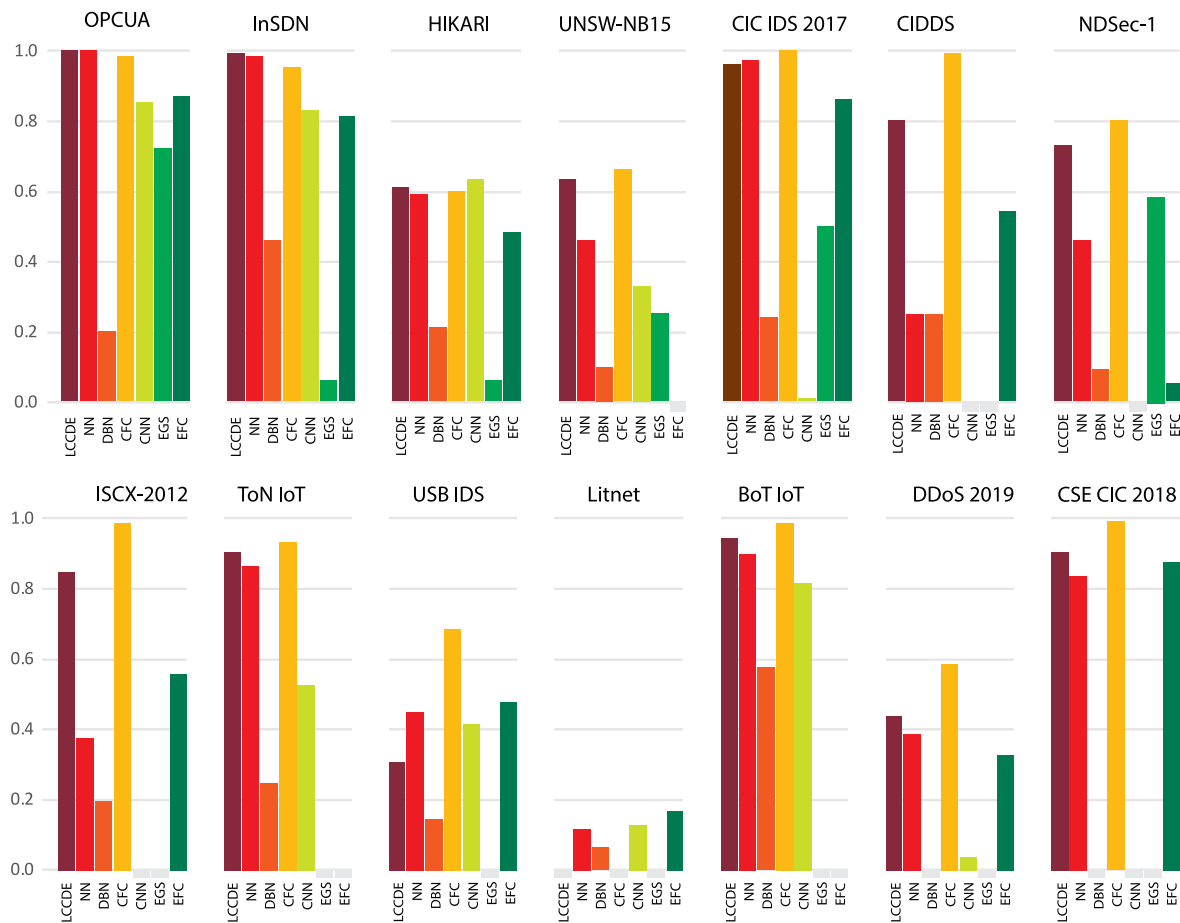


Fig. 6 Macro F1 Scores for OPCUA, InSDN, Hikari, UNSW-NB15, CIC IDS 2017, CIDDS, NDSec-1, ISCX-2012, ToN IoT, USB IDS, Litnet, BoT IoT, DDoS 2019 and CSE CIC 2018 datasets. Gray color is shown for methods that Did Not Finish

Alongside, LCCDE [8] finished 13 out of 14 datasets and ranked close to CFC [87]. The non-linear nature of the ensemble model allows the method to adapt to different contexts. Nevertheless, neither CFC [87] nor LCCDE [8] could complete the Litnet [12] dataset, which displays a real scenario. The method shows the best results as the amount of instances is low, and displays a stable performance for the remaining datasets. Figure 8 indicates a median performance worse than CFC [87] but better than the other methods. Although this method was tested without the original SMOTE [82] pre-processing technique, it is still resistant to class imbalance and even robust as the datasets grow.

Considering all the datasets in which LCCDE [8] or CFC [87] were ranked first or second, they dominated at least half of the cases. An essential characteristic that these two methods include is the integration of categorical features as supported attribute types. A summary of the methods' characteristics and average Macro F1 score can be found in Table 20.

NN [80] method, the simplest of the Deep Learning architectures included in this study was close to LCCDE [8] and CFC [87] on at least half of the cases. Although it was never the best method, its average Macro F1 score is followed by EFC [73]. Nevertheless, as Fig. 6 shows its median performance is below 0.6, almost 20% below LCCDE [8]. It is important to highlight its training speed due to the shallow architecture. This algorithm could generate a model for every dataset disregarding the size and showcasing its robustness but, as every neural network only works with numerical features and would need dataset pre-processing to handle categorical variables.

EFC [73] could finish successfully 12 out of 14 datasets with a wide range of results. Being really close in performance to CNN [59], it has the advantage of being able to work with different types of datasets such as categorical and mixed sets while showing a mean Macro F1 score higher than CNN [59] and close to NN [80]. As a method that is based on a new technique, it still lacks testing on different datasets.

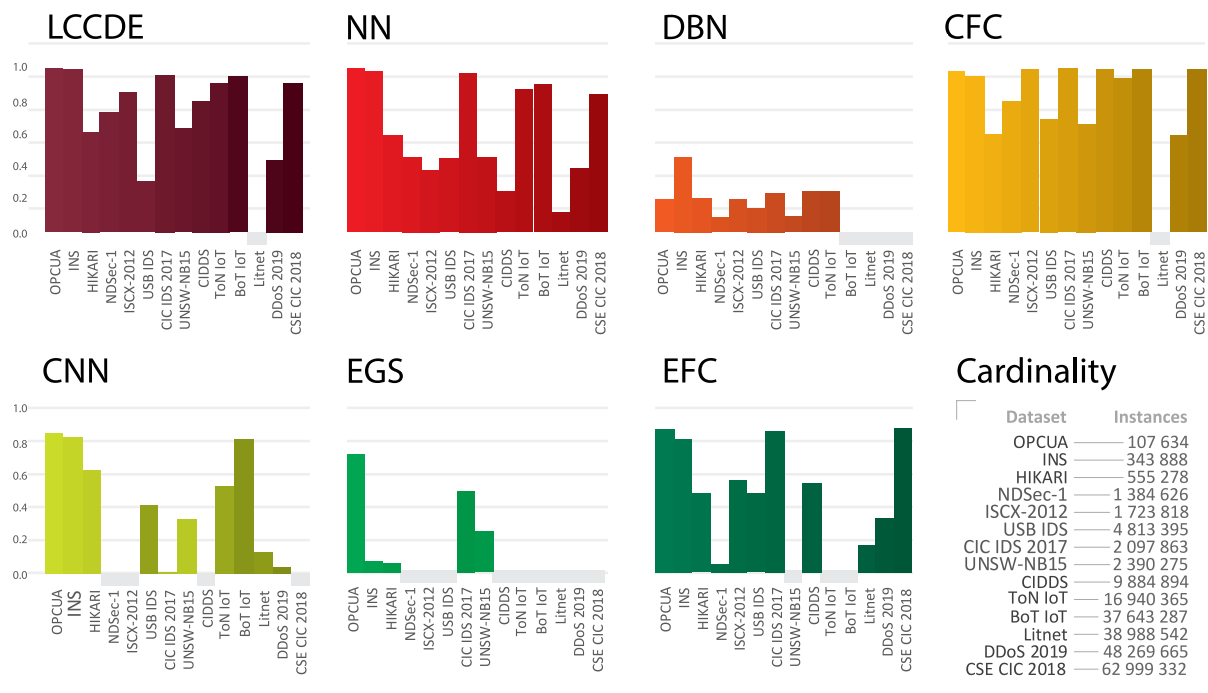


Fig. 7 Macro F1 Scores for LCCDE, NN, DBN, CFC, CNN, EGS, EFC methods ordered by dataset size. Gray color is shown for methods that Did Not Finish

There were some issues on memory handling that could not be avoided. Nevertheless, it provides minimal configuration and fast setup.

A significant disadvantage of the CNN [59] method is the amount of features it requires on the dataset. Although in real-world scenarios is not a problem as industry standard PCAP extractors output more than 30 [41, 47], some cases

could not be completed. The method's performance on the standard CIC IDS 2017 [10] was deficient, especially when measured against the top 4 methods. CNN [59] results could not be reproduced due to dataset pre-processing performed on the original paper. By sampling the attacks, the benchmark is different and the specific tuning done avoids a fair comparison. This method was the best on the recent Hikari [17]

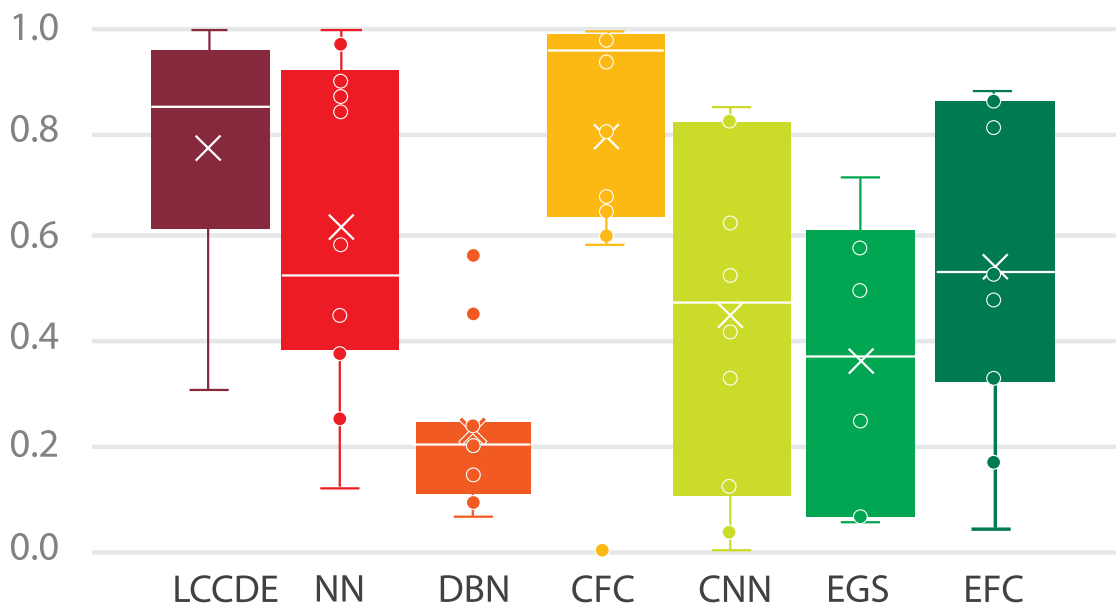


Fig. 8 Whisker plot chart for Macro F1 Scores by Algorithm

Table 20 Comparison summary with LCCDE, NN, DBN, CNN, EGS, EFC, CFC methods regarding categorical handling, imbalance resistance, test completion rate out from 14 datasets, number of out-of-memory errors on 14 datasets and averaged Macro F1 Score

Method	Characteristic				
	Categorical Handling	Imbalance Resistant	Completion Rate	Out-of-memory Errors	Average Macro F1 Score
LCCDE	Yes	No	0.93	1	0.78
NN	No	No	1.00	0	0.62
DBN	No	No	0.86	2	0.23
CFC	Yes	Yes	0.93	1	0.86
CNN	No	No	0.71	1	0.46
EGS	Yes	No	0.43	3	0.36
EFC	Yes	No	0.79	3	0.55

dataset, which has the same netflow flow standard as CIC IDS 2017 [10], highlighting that even with the same dataset structure, the scenarios are meant to be solved differently. As the plot on Fig. 8 shows, CNN's median Macro F1 score is close to 0.5, close to NN [80], EFC [73] and EGS [9]. However, CNN also exhibits a high variance.

The two final methods, EGS [9] and DBN [81] display the most complex architectures and stand out as state-of-the-art techniques. While they did not achieve particularly high Macro F1 scores or even finish some the cases, this cannot correlate directly to bad performance. In any case, these methods took the most extended time to train and, in some cases, would be even timed out by the settings. By design, DBN [81] shows flexibility and options to be tailored to any dataset. Nevertheless, contrasting with reported results on methods such as the CIC IDS 2017 [10] dataset, DBN [81] could not perform as well as initially expected due to pre-processed and merged classes. In the original work, the dataset was reduced to 6 labels instead of 12, disallowing any direct comparison. In its paper, EGS [9] authors report fewer instances for the same datasets that we obtained from the sites, making it difficult for a standardized test. We highlight the importance of keeping the benchmarks as close as possible to the original.

Datasets Performance Analyzing the datasets, we notice that the Imbalance Ratio on many of them is above 1000 as can be seen on Table 1. Nevertheless, the NORMAL or BENIGN label is not always appointed as the majority one. Researchers on datasets such as USB IDS [37] or NDsec-1 [35] have found ways to capture traffic of interest. Studying their methods is essential to understanding the architectures and generation of benchmarks.

As reported in other studies, the PCAP extraction process still poses a differential aspect on dataset generation and evaluation. The Netflow versions used in this study such as Sarhan's et al. [33] UNSW-NB15, ToN IoT, and BoT IoT show worse performance than the reported non-Netflow counterparts [11, 56, 58]. Researchers and practitioners must

carefully select the datasets based on their own process and target industry standards. General methods can contrast them to measure their sensitivity to similar architectures but different attributes.

OPCUA [31] and InSDN [32] datasets were solved by at least 3 methods with a Macro F1 score of 0.95 and a mean performance of 0.803 and 0.727. On the other hand, Hikari [17] dataset had a mean performance of 0.454 with more instances and less imbalance ratio. All of them are recent and developed towards contemporary architectures. The methods that ranked first on the two former ones were outperformed by CNN [59] on the latter, as can be seen on Table 19. Therefore, testing the methods on these shows different characteristics and opportunities, even in ideal network conditions and scenarios.

Datasets such as UNSW-NB15 [33] or NDsec-1 [35] are still a challenge even though they have been available for more than 6 years. These have a mean Macro F1 score of 0.405 and 0.452 respectively. On top of this, UNSW-NB15 [11] shows an Imbalance Ratio of 13,995.256 with 9 classes, and NDsec-1 [35] has 915,613.00. A common characteristic between UNSW-NB15 [11] and NDsec-1 [35] is that both were labeled through the use of signature-based IDS systems, as an operative data scenario would be. Close to Litnet [12], although not wholly connected to real networks, these contexts obtained low Macro F1 scores.

CIC IDS 2017 [10] and CSE CIC 2018 [39] had an average Macro F1 score greater than 0.80 if CNN [59] and EGS [9] are not considered. Although an essential difference between them is the number of instances, these two scenarios pose large class imbalances. As these datasets are the standard, methods that are tested on them are expected to score above these numbers. Nevertheless, the results of the methods' characterizations are not entirely conclusive as it is not clear if the tests were done on the erroneous CIC IDS 2017 [10] and CSE CIC 2018 [39], or on the corrected ones. Other datasets such as Hikari [17], UNSW-NB15 [11] or NDsec-1 [35], should also be tried when developing proposals as these pose other scenarios.

ToN Iot and Bot IoT on Sarhan versions [33] implement Netflow standard features. They show relatively high average Macro F1 scores but do present issues for the algorithms. A significant feature set and tens of millions of registers are included in these scenarios, oriented towards IoT networks. The methods tried on these datasets are challenged by the high volume of data that needs to be ingested and could be used to optimize the technical requirements of solutions.

Litnet [12] dataset was particularly challenging for every method due to its high imbalance ratio and high memory requirements. This was the only benchmark that CFC [87] could not complete and considering that it is the only dataset with real traffic, highlights the importance of testing on this type of scenario.

As datasets have different characteristics but could be generalized with a network flow standard, a benchmark combining 2 or 3 is not widely researched. The datasets reported in this paper are compatible in terms of features, but no study has been done in terms of the separability and compatibility of classes.

5.2 Implementation implications

While the main findings show facts such as CFC or LCCDE being able to work in different contexts and that the different datasets are needed to test methods thoroughly, there are other perspectives that need to be highlighted. The general NIDS process starts with extracting the flows from the network. This means that extra hardware and software are needed to recover the connection logs and generate records. It is challenging to set up an infrastructure capable of managing thousands of sessions each minute and achieving intrusion detection in real-time. NIDS focusing on packets avoids this configuration by receiving the same data as the rest of the network. On top of this, flows cannot be completed until the connection is finished, i.e. after the possible damage has been caused.

Provided that the network flows can be generated, the problem of setting up a common record structure needs to be addressed. Although there are proposals such as NetFlow [33, 47] or CICFlow [41], datasets show that there is no standard, having flows with only 8 attributes and up to 82. To be able to share traffic logs and test methods faster, network administrators, researchers and practitioners need to have compatible formats that do not imply downloading terabytes of PCAP files. Due to issues like PCAP file sharing, network flow generation, and different formats, training and testing models for NIDS takes time.

Training on datasets such as the 14 presented in this work is faster as the pre-processing has already been made and is ready for use in a pickle format. Practitioners can use this information to test their methods or pick some of the ones pre-

sented here. Either way, this provides a quicker way to start and deploy. Also, these datasets provide different network arrangements and contexts, being able to test on Software Defined Networks, IoT environments, Corporate Networks or even Metropolitan Networks. This needs to be chosen carefully as the attacks vary. When the new method is ready, it will not only define the type of threats, but also the best network flow standard to be used.

NIDS technology is being developed simultaneously with ML methods. Being able to test Convolution Neural Networks or Graph Neural Networks allows researchers to further investigate the relevant patterns in the data. Due to the speed needed on the field, some models are not suitable for day-to-day updates. EGS Method took the longest average time to train, it needed more configuration tailoring and it was not the best in terms of F1-Score. While we do not discourage its use, specific hardware, time resources, and smaller network flows make its use more challenging than other solutions.

On the other hand, LCCDE was designed to run on low-end hardware as it was oriented to systems running on vehicles. It is based on tested software components with ensemble methods and needs little to no configuration. Having little requirements both in hardware and software, makes it a feasible solution when resources are scarce. Due to its characteristics, is a technique that can be implemented on Edge, working best with limited hardware.

DBN method was the lowest performer but this should not prevent implementations. Configuration for this solution is challenging, can be modified from a single file, and needs its own optimization process. Deep Belief Networks such as this one imply using a large volume of resources [67] so it should be avoided on limited hardware. The configuration and performance on this work can provide a starting point for the architecture search when trying the method as the original results showed potential but used less columns and a different version of CIC IDS 2017 [10].

When developing new methods or testing a specific one, CFC [87] shows the best potential for any network. Although the F1-scores were higher than all of the other solutions in different types of scenarios, it took too much time to train on Litnet [12] and did not finish. Either way, it can use GPUs, manages categorical data automatically, includes model backups, is ready for usage from the command line, supports few-shot learning, and its default options work for almost any type of context.

If neural network solutions cannot be implemented on the networks in situations such as low-end hardware, edge-computing, or even in scenarios with hardware limitations, EFC and LCCDE are two feasible candidates. Being able to work with only the CPU, these methods are already available in the scikit-learn estimator standard and can integrate with other solutions. Practitioners would be warned that EFC is

still under development as a new module, does not use any other components and at the time of writing this work, it had some memory issues.

5.3 Comparison to other studies

Studies have been made, comparing different solutions, stating the technological key characteristics and outlining the attacks but they still fail to develop a benchmark, provide initial materials for new research to start with, and expand information on existing datasets. One specific issue that we highlight is that new solutions and surveys include CIC IDS 2017 [10] but none of them state the error on the network flows discovered by Engelen et al. [16], Rosay et al. [51], Liu et al. [52] and Lanvin et al. [53].

Verma et al [105] provide a SWOT Analysis on State-of-the-Art NIDS solutions covering 43 different methods, with 13 of them being non-deep learning solutions. The survey provides a discussion of the different proposals in the last 8 years but does not expand on datasets and different scenarios. The list of current algorithms includes the datasets in which they were tested and refers to previous work in [106] where there is more information about 9 datasets. Three of the presented datasets contain information previous to the year 2001 and some of them are inaccessible. By combining the two papers, the authors provide a current context of the solutions but an outdated perspective of the datasets. Our work fulfills that gap by providing the pre-processed usable datasets - for the first time among research - reference to open algorithms, and a practitioner's first test of NIDS methods.

Abdulganiyu et al [67, 107] presented a systematic literature review for NIDS in which they reviewed more than 50 methods and 13 datasets covering solutions signature-based, AI-based, and hybrid. Being one of the most extensive works in the field, it does not discuss the datasets thoroughly and presents the methods with the original paper metric. Besides this, they report that out of the 13 datasets stated, 2 of them are already outdated and do not provide links to them. Our work presents 14 downloadable datasets and reports 7 methods with the performance metrics on the same scenarios.

Kilincer et al [93] report a comparative study between 8 general methods covering variations of SVM, Decision Trees, and KNN techniques. By providing the results of the models in 5 datasets, the work presents a general view of the different scenarios, in contrast to other surveys. With results comparable to ours, the authors are not proposing any model but rather a methodology to work with various algorithms. Their scores are compared to literature results on cases where the measure is available. As there is no standard metric, the comparison is incomplete. Our study extends Kilincer [93] proposal and deepens it by testing more datasets and 7 different algorithms, all being freely available.

Ahmetoglu and Das [18] feature machine learning, deep learning, feature selection, and test methods in a study that compares literature results among different techniques. The work reports 10 datasets, 3 of which are already outdated with attacks previous to 2000, and 1 is directed towards Host Intrusion Detection Systems. The models' results are contrasted based on literature findings failing to provide a common benchmark. From the 42 methods presented, 14 are only tested on NSL-KDD [14] or KDD99 [15] only, leaving doubts about their applicability to current threats. Our study compares the results of the methods in a common benchmark, helping to close the gap between the literature results and the practitioners view.

Ring et al [23] provide a report on cybersecurity datasets up to 2017. By presenting the methodology to create a dataset and the different formats, they introduce 34 datasets highlighting which ones are publicly available and providing the download links. Half of the datasets are publicly available, labeled, and recent. The rest are old, private or even unlabeled and provided in PCAP format. This is one of the most complete works on NIDS datasets but lacks a final comparison of the methods that have used the datasets.

All the studies presented cover an important area in the field, whether it is the technique explanation, the list of methods that have been proposed, and the datasets available worldwide. Nevertheless, the vast majority do not perform any study based on the same datasets or implement the researched methods to provide a benchmark. Our study closes that gap by searching for freely open methods and testing them over 14 downloadable datasets, providing a general perspective but also a starting point for new research.

5.4 Implications for new practitioners and researchers

Generating a useful dataset is challenging as real networks generate thousands of sessions per minute, that will not necessarily show useful information. Moreover, the amount of storage for the information that is generated scales rapidly in terms of hardware needs. On the other hand, NIDS intelligent methods need their own resources to extract patterns and test the performance, particularly time and hardware. New practitioners and starting researchers have a challenging task when introducing themselves to the field, whether to implement a system or to experiment and develop other solutions.

As new datasets are being generated, their distribution needs to be communicated. Different scenarios, contexts, and network infrastructures are imprinted on each dataset. New users must study the characteristics of the information and start their own path as needed. Unfortunately, this can take months up to the moment in which they can decide. In the meantime, attacks and threats are still on the rise.

Our work targets these difficulties. We provide a deeper understanding of the different available datasets and provide initial guidance on which one to select. Whether it is a Software Developed Network, an IoT infrastructure, or a Corporative Network, there is a research team that has developed an example. However, once the datasets are downloaded there is still work to be done.

New practitioners will find a long way to go from knowing which dataset is better to start training with to implementing their own solutions. Many of these datasets still need some pre-processing to be done as different network flow extractors are implemented, obtaining alternative data structures, file formats, null values, and data types. Many of them will require a large download in terms of gigabytes or terabytes. After performing steps to gather data and transform it into a useful format, practitioners can start training the models.

Once models are generated, the implementation on networks need a robust PCAP extractor and analyzer working to translate sessions to network flows. This is utterly important as the models are trained on the aggregated forms of communication. If this is not possible, the system cannot identify the threats timely, rendering a defenseless network. Methods such as LCCDE or EFC, designed for small networks as not focused on high volume scenarios. Systems with resources such as Tensor Processing Units or Graphics Processing Units can leverage neural networks to work quickly and accurately.

Certain algorithms and solutions need specific hardware to run. LCCDE or EFC methods can be implemented on almost any computer, but CFC, DBN, or EGS are designed for servers and high-end computers with advanced resources. Choosing the method to compare with according to the problem requirements, can accelerate or brake the project of developing an intelligent NIDS.

To ease the process and bootstrap new research, this work presents 14 datasets, ready in pickle format, allowing a straightforward easy way to start developing solutions in Python. Distributed as a pandas object, the file can be further exported to other formats such as *arff* or *parquet*. All the data in the files has already been pre-processed, so researchers can start immediately running experiments.

To help new researchers, this work presents and links 7 different Python methods covering ML and deep learning options implemented as Jupyter notebooks or ready-to-use modules. Researchers can rest assured the baseline algorithms will work, and they can choose which ones to use as the characteristics have already been outlined.

5.5 Limitations and general recommendations

AI for NIDS has been explored recently. ML and neural networks have been implemented to detect patterns in network flows and alert administrators over unexpected guests in a

network, meaning a threat to the rest of the users. Starting new research in this field is challenging due to outdated or unprocessed datasets and unavailable comparable methods. This work presents 14 datasets and corresponding download links, ready to train an ML method. To provide a benchmark, we tested 7 algorithms and compared the models' performance in the 14 datasets to help practitioners and researchers in choosing a competitive method or an adequate dataset.

Although we diligently performed every step possible to achieve a complete benchmark, there were some limitations that should be addressed in further research. Datasets found were not obtained from PCAP files, but from already labeled network flows. We acknowledge that the options available grow when obtaining the PCAP files directly. Nevertheless, doing so would have required to label the traffic, rendering a different focus on the research.

Our study was designed to use only free and open-source algorithms. The objective was to give tools to new practitioners and researchers to start their work as quickly as possible, though other algorithms could have been implemented by following the proposals in the paper. Once the algorithms were adapted to one common interface, they were trained with each dataset. In this regard, the hyperparameter search was not extensive. For example, DBN can be configured with numerous architectures from which we selected only the ones specifically proposed on the original paper. In the same way, we tried to keep the batch numbers low for a better approximation but the epochs can be further expanded. Methods such as EFC had just one parameter. The source code for each algorithm had default options which we tried to follow closely. A larger hyperparameter search, potentially focusing on a subset of algorithms, is yet to be tested.

To limit the study, we did not try mixed systems with two or more models focusing on specific classes. Moreover, each dataset was used as closely as possible to the version available online which we downloaded. Each dataset was only processed for the null values and data formats. Only the versions with the same network flow standard could be tested as a bigger mixed set.

The study presented here is not designed to cover every attacks on networks, but rather on the datasets available. We understand that this does not cover the whole range of modern threats, but rather presents the current situation of testbeds. As stated earlier, the never-ending nature of attacks evolution keeps the datasets classes incomplete and needing new scenarios.

Other pre-processing techniques such as balancing the datasets were not applied. In effect, LCCDE included resampling techniques on the original code. To provide a fair comparison we avoided this pre-processing. Categorical variables were only passed to the methods that could directly handle them internally and we did not execute any encoding technique such as "One Hot Encoding" or "Embedding".

Numerical columns were not explicitly normalized from the beginning. This was only applied when the algorithm internal steps consider it.

While we acknowledge these limitations, there are some recommendations that should be followed when working with these datasets and NIDS methods.

As stated by [108], NSL-KDD [14] or KDD99 [15] datasets “include legacy malware that no longer attacks current network systems”. Newer datasets such as the ones presented here cover the same attacks such as U2R, Probing or DoS with recent malware, network architectures and protocols. Nevertheless, some of the attacks covered in the network flows are underrepresented with very specific patterns for them. This creates a gap that needs to be filled by the datasets to come.

New practitioners and researchers should get used and know well the dataset that they are using and they should also know how it was built and where does it come from. This initial understanding of the dataset is critical. Software Defined Networks compared to IoT have different characteristics and threats. Even when considering the same network flow standard, methods’ performance can differ greatly. For example, DDoS 2019 and CSE CIC 2018 share the same features but algorithms struggle to keep similar performance metrics values on both.

When working with NIDS datasets there are two main cases that should be considered: anomaly detection and multi-class prediction. Practitioners should take into account both scenarios in order to identify patterns on a specific threat and then compare with the multi-class case scenario. Systems with many binary class models can be built instead of a single multi-class. Nevertheless, there are threats that are similar and the methods should be able to identify and distinguish between them. Deep learning models will consume more time and resources especially when dealing with the biggest datasets. However, we recommend that researchers first consider the new methods on the 14 cases proposed here as that test will potentially unveil important algorithms and models characteristics.

Data is expected to be naturally imbalanced and this must be considered in the design of new methods. CFC integrates a solution to this by means of Contrastive Learning and it showed a better average performance than the alternatives. Several different performance metrics should also be considered when assessing these solutions [109].

We also consider that, when implementing an intelligent NIDS researchers should consider a signature-based system simultaneously. It will recognize attacks initially and will allow a period of learning to the AI based. Both solutions will capture traffic of interest and help protect the network providing redundancy. We must highlight that a production system

needs a PCap Extractor or other software that generates network flows next to the NIDS solution. When deciding on these systems, standards such as Network Flow or CICFlow will benefit the practitioners to compare solutions.

This research is meant to be a single point of entry to datasets and methods, for later specialization and development. We have no control over new methods developed, but we can state the scenarios to test them and provide some easy-to-implement methods.

Key Takeaways

The broad experiments carried out provided the following insights:

- **Missing standard benchmark and methodology.** Attacks are evolving and generate difficult conditions for up-to-date data. A benchmark is crucial to allow a fair comparison of solutions but is still missing. Besides the benchmark, a methodology for different network architectures and packet capture is needed. The PCAP Extraction process must be designed with already available common feature sets so methods need less specific configuration.
- **Datasets involving different architecture and attack configurations.** To better understand the situation in which a certain IDS is effective and efficient, they must be tested on a wider range of scenarios. Different datasets help in evaluating the newest methods opening the possibility for solutions that work in broader settings.
- **Need to report method’s complexity.** IDS are implemented on different settings that include desktops and laptops as well as automotive Electronic Control Units. Researchers can target methods that rely on different ML techniques. In this way, users can choose the most suitable ones for their settings. To correctly evaluate this, the complexity of the methods needs to be systematically reported.
- **Real and synthetic datasets are needed.** Attacks happen at different rates. Samples taken from real networks do not imply every possible but do show required patterns. Using real datasets is important. However, synthetic datasets allow researchers to capture specific conditions and even study commonly unseen configurations. Both real and synthetic datasets should be analyzed when comparing solutions in this context.
- **Imbalance on datasets is implied and must be considered.** Any method trying to capture intrusion patterns needs the capability to deal with infrequent attack samples. Even on available synthetic datasets, attacks are not equally distributed. In certain cases, the benign traffic

is the minority class. Methods need imbalance-resistant designs.

- **Methods must deal with large volumes of data and near real-time detection.** As the attacks evolve, retraining is carried out often. The amount of data captured by network devices is growing as more devices are being added to networks. Recent datasets have more than dozens of millions of records. New solutions that can ingest the data producing lightweight accurate models are desired.
- **Methods based on contrastive learning and non-linear patterns such as Decision Trees show desired average performance.** For the development of new techniques, solutions such as CFC or LCCDE are easily implemented and allow a state-of-the-art comparison. Researchers should take into account these baselines when evaluating new solutions.

6 Conclusion

Network Intrusion Detection Systems are installed on networks to detect and issue alarms for unwanted requests. There are two main ways of analyzing the traffic. Packet-based IDS identifies threats by reading each information transferred between two hosts. On the other hand, Flow-based IDS aggregates the communication between two hosts and analyzes attacks based on the completed session. In this paper, we study 7 network flow IDS algorithms on 14 different datasets. Our main objective was to show relevant challenges that were previously unreported in terms of datasets and algorithms. As an outcome of our efforts to compile a large number of datasets for this predictive task, we provide a large standardized benchmark with recent 14 datasets. We provide all the details that enable the assessment and direct comparison of results of new algorithms in these datasets at <https://github.com/UOttawa-Cyber-Range-Research/GuardiansOfTheNetwork>. After a detailed analysis of the datasets, we carry out a large set of experiments on a broad range of scenarios. While the main focus was not to identify one best method, there were 2 algorithms that stood out for their imbalance resistance capabilities and stability.

CFC [87] algorithm showed the best Macro F1 score overall followed closely by LCCDE [8] and a Neural Network Architecture based on Fully Connected Layers [80]. While only one method could finish the experiments on all the datasets, results highlight that datasets such as OPCUA [31], InSDN [32], CIC IDS 2017 [10], Bot IoT [58], and CSE CIC 2018 [39] had at least 4 algorithms achieving more than 0.80 of average Macro F1 score, thus exhibiting a high per-

formance. The first two are small datasets, the next two are medium-sized benchmarks, and the final is a large dataset, suggesting a combination of these databases and algorithms for future research.

The lowest average Macro F1 score was reported on the Litnet [12] dataset. This benchmark is the closest to a real scenario as it was built with traffic on an operating metropolitan network. By showing these results, we outline the need for more datasets such as this one containing not only realistic imbalanced samples but also daily network operations.

Methods based on Restricted Boltzmann Machines [81] or Graph Neural Networks [9] were also tested. Unfortunately, due to the architecture search or the amount of time they needed to finish, these techniques achieved low performance when compared to the remaining alternatives. While we do not have evidence that these techniques are unpromising, the complexity of their design makes testing them challenging for researchers and practitioners.

Selecting the benchmark to test a method is still a design challenge. Considerations on specific scenarios as whether the method is oriented towards the IoT, Corporate Networks, or specific attacks need to be studied thoroughly. We did not find a specific subset that could be selected to test every IDS aspect, though, there are attempts to consolidate benchmarks [33]. Nevertheless, due to the PCAP Extraction software used, the different set of features is still an ongoing challenge.

With this study, researchers can locate and decide datasets and methods to test against and start new proposals faster with focused scenarios. As some of the datasets had issues, we reported their original and modified versions using the most updated ones. Every algorithm implementation used was freely available on GitHub⁴⁴ and the corresponding links can be found throughout the document. There are still areas to explore regarding the datasets and the steps before, such as PCAP Extraction and Network Flows Feature generation.

Author Contributions Jose Carlos Mondragon: Conceptualization, Methodology, Investigation, Writing - original draft, Software. Paula Branco: Conceptualization, Methodology, Project Administration, Resources, Validation, Supervision, Writing - review & editing, Funding Acquisition. Guy-Vincent Jourdan: Conceptualization, Resources, Supervision, Validation, Funding Acquisition, Writing - review & editing. Andres Eduardo Gutierrez-Rodriguez: Supervision, Validation, Writing - review & editing. Rajesh Roshan Biswall: Supervision, Validation, Funding Acquisition, Writing - review & editing.

Funding This work was supported by the Mitacs Globaling Research Award - for research in Canada [Funding Internship Ref.: FR97683].

Code and Data Availability All the data and code used in experiments is available at: <https://github.com/UOttawa-Cyber-Range-Research/>

⁴⁴ <http://www.github.com>

GuardiansOfTheNetwork

Editorial Policies for:

Springer journals and proceedings: <https://www.springer.com/gp/editorial-policies>

Nature Portfolio journals: <https://www.nature.com/nature-research/editorial-policies>

Scientific Reports: <https://www.nature.com/srep/journal-policies/editorial-policies>

BMC journals: <https://www.biomedcentral.com/getpublished/editorial-policies>

Declarations

Competing Interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethical and Informed Consent for Data Used This article does not contain any studies with human participants or animals performed by any of the authors.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Dhanya KA, Vajipayajula S, Srinivasan K, Tibrewal A, Kumar TS, Kumar TG (2023) Detection of network attacks using machine learning and deep learning models. *Procedia Comput Sci* 218:57–66 <https://doi.org/10.1016/j.procs.2022.12.401>
2. Molina-Coronado B, Mori U, Mendiburu A, Miguel-Alonso J (2020) Survey of network intrusion detection methods from the perspective of the knowledge discovery in databases process. *IEEE Trans Netw Serv Manage* 17(4):2451–2479. <https://doi.org/10.1109/tnsm.2020.3016246>
3. Macas M, Wu C, Fuertes W (2022) A survey on deep learning for cybersecurity: Progress, challenges, and opportunities. *Comput Netw* 212:109032 <https://doi.org/10.1016/j.comnet.2022.109032>
4. Pawlicki M, Kozik R, Choraś M (2022) A survey on neural networks for (cyber-) security and (cyber-) security of neural networks. *Neurocomput* 500:1075–1087. <https://doi.org/10.1016/j.neucom.2022.06.002>
5. Fu J, Wang L, Ke J, Yang K, Yu R (2022) GANAD: a GAN-based method for network anomaly detection. <https://doi.org/10.21203/rs.3.rs-2081269/v1>
6. Santos RR, Viegas EK, Santin AO, Cogo VV (2022) Reinforcement learning for intrusion detection: More model longness and fewer updates. *IEEE Trans Netw Serv Manage*, 1–1. <https://doi.org/10.1109/tnsm.2022.3207094>
7. Li B, Springer J, Bebis G, Hadi Gunes M (2013) A survey of network flow applications. *J Netw Comput Appl* 36(2):567–581
8. Yang L, Shami A (2022) IDS-ML: An open source code for intrusion detection system development using machine learning. *Softw Impact* 14:100446. <https://doi.org/10.1016/j.simpa.2022.100446>
9. Lo WW, Layeghy S, Sarhan M, Gallagher M, Portmann M (2022) E-graphsage: A graph neural network based intrusion detection system for iot. In: *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, pp 1–9. IEEE
10. Sharafaldin I, Lashkari AH, Ghorbani AA (2018) Toward generating a new intrusion detection dataset and intrusion traffic characterization, 108–116. <https://doi.org/10.5220/0006639801080116>
11. Moustafa N, Slay J (2015) UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). <https://doi.org/10.1109/milcis.2015.7348942>
12. Damasevicius R, Venckauskas A, Grigaliunas S, Toldinas J, Morkevicius N, Aleliunas T, Smuikys P (2020) Litnet-2020: An annotated real-world network flow dataset for network intrusion detection. *Electron* 9(5). <https://doi.org/10.3390/electronics9050800>
13. Apruzzese G, Pajola L, Conti M (2022) The Cross-evaluation of Machine Learning-based Network Intrusion Detection Systems. *IEEE Trans Netw Serv Manage (IEEE TNSM)*. IEEE
14. Tavallaei M, Bagheri E, Lu W, Ghorbani AA (2009) A detailed analysis of the KDD CUP 99 data set <https://doi.org/10.1109/cisda.2009.5356528>
15. California I. KDD Cup 1999 Data. University of California, Irvine. <https://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>
16. Engelen G, Rimmer V, Joosen W (2021) Troubleshooting an intrusion detection dataset: the CICIDS2017 case study. <https://doi.org/10.1109/spw53761.2021.00009>
17. Ferriyan A, Thamrin AH, Takeda K, Murai J (2021) Generating network intrusion detection dataset based on real and encrypted synthetic attack traffic. *Appl Sci* 11(17). <https://doi.org/10.3390/app11177868>
18. Ahmetoglu H, Das R (2022) A comprehensive review on detection of cyber-attacks: Data sets, methods, challenges, and future research directions. *Internet of Things* 20:100615. <https://doi.org/10.1016/j.iot.2022.100615>
19. Jmila H, Khedher MI (2022) Adversarial machine learning for network intrusion detection: A comparative study. *Comput Netw* 214:109073. <https://doi.org/10.1016/j.comnet.2022.109073>
20. Sewak M, Sahay SK, Rathore H (2022) Deep reinforcement learning in the advanced cybersecurity threat detection and protection. *Inf Syst Frontier*. <https://doi.org/10.1007/s10796-022-10333-x>
21. Ahmad Z, Khan AS, Shiang CW, Abdullah J, Ahmad F (2020) Network intrusion detection system: A systematic study of machine learning and deep learning approaches. *Trans Emerg Telecommun Technol* 32(1). <https://doi.org/10.1002/ett.4150>
22. Antunes M, Oliveira L, Seguro A, Verissimo J, Salgado R, Murteira T (2022) Benchmarking deep learning methods for behaviour-based network intrusion detection. *Inf* 9(1):29. <https://doi.org/10.3390/informatics9010029>
23. Ring M, Wunderlich S, Scheuring D, Landes D, Hotho A (2019) A survey of network-based intrusion detection data sets. *Comput Secur* 86:147–167
24. Sarker IH (2022) Machine learning for intelligent data analysis and automation in cybersecurity: Current and future prospects. *Annals of Data Sci*. <https://doi.org/10.1007/s40745-022-00444-2>
25. Thakkar A, Lohiya R (2021) A survey on intrusion detection system: feature selection, model, performance measures, application perspective, challenges, and future research directions. *Artif Intell Rev* 55(1):453–563. <https://doi.org/10.1007/s10462-021-10037-9>
26. Gharib A, Sharafaldin I, Lashkari AH, Ghorbani AA (2016) An evaluation framework for intrusion detection dataset. In:

- 2016 International Conference on Information Science and Security (ICISS), pp 1–6 . <https://doi.org/10.1109/ICISSEC.2016.7885840>
27. Aldweesh A, Derhab A, Emam AZ (2020) Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues. *Knowl-Based Syst* 189:105124. <https://doi.org/10.1016/j.knosys.2019.105124>
28. Adawadkar AMK, Kulkarni N (2022) Cyber-security and reinforcement learning — a brief survey. *Eng Appl Artif Intell* 114:105116. <https://doi.org/10.1016/j.engappai.2022.105116>
29. Oracle (2017) 7 securing ports. Oracle. https://docs.oracle.com/cd/E89228_03/otn/pdf/install/html_edmsc/output/chapter_6.htm
30. Ortigosa-Hernández J, Inza I, Lozano JA (2017) Measuring the class-imbalance extent of multi-class problems. *Pattern Recognit Lett* 98:32–38. <https://doi.org/10.1016/j.patrec.2017.08.002>
31. Pinto R (2020) M2M using OPC UA. *IEEE Dataport*. <https://doi.org/10.21227/yhcv-6c68> . <https://dx.doi.org/10.21227/yhcv-6c68>
32. Elsayed MS, Le-Khac N-A, Jurcut AD (2020) Insdn: A novel sdn intrusion dataset. *IEEE Access* 8:165263–165284. <https://doi.org/10.1109/ACCESS.2020.3022633>
33. Sarhan M, Layeghy S, Portmann M (2021) Towards a standard feature set for network intrusion detection system datasets. *Mobile Netw Appl* 27(1):357–370. <https://doi.org/10.1007/s11036-021-01843-0>
34. Ring M, Wunderlich S, Grödl D, Landes D, Hotho A (2017) Flow-based benchmark data sets for intrusion detection. In: *Proceedings of the 16th European Conference on Cyber Warfare and Security (ECCWS)*, pp 361–369. ACPI, ???
35. Beer F, Hofer T, Karimi D, Bühler U (2017) A new attack composition for network security. In: Müller P, Neumair B, Raiser H, Dreö Rodosek G (eds) 10. DFN-Forum Kommunikationstechnologien, pp 11–20. Gesellschaft für Informatik e.V., Bonn
36. Shiravi A, Shiravi H, Tavallaee M, Ghorbani AA (2012) Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *Comput Secur* 31(3):357–374. <https://doi.org/10.1016/j.cose.2011.12.012>
37. Catillo M, Del Vecchio A, Ocone L, Pecchia A, Villano U (2021) Usb-ids-1: a public multilayer dataset of labeled network flows for ids evaluation. In: 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W), pp 1–6. <https://doi.org/10.1109/DSN-W52860.2021.00012>
38. Sharafaldin I, Lashkari AH, Hakak S, Ghorbani AA (2019) Developing realistic distributed denial of service (ddos) attack dataset and taxonomy. In: 2019 International Carnahan Conference on Security Technology (ICST), pp 1–8. <https://doi.org/10.1109/CCST.2019.8888419>
39. New Brunswick U. CSE-CIC-IDS2018 on AWS. Canadian Institute of CyberSecurity. <https://www.unb.ca/cic/datasets/ids-2018.html>
40. Farhady H, Lee H, Nakao A (2015) Software-defined networking: A survey. *Comput Netw* 81:79–95. <https://doi.org/10.1016/j.comnet.2015.02.014>
41. Lashkari AH. CICFlowMeter (formerly ISCXFlowMeter). Canadian Institute for Cybersecurity. <https://www.unb.ca/cic/research/applications.html>
42. Maciá-Fernández G, Camacho J, Magán-Carrión R, García-Teodoro P, Therón R (2018) Ugr 16: A new dataset for the evaluation of cyclostationarity-based network idss. *Comput Secur* 73:411–424. <https://doi.org/10.1016/j.cose.2017.11.004>
43. Applied Internet Analysis C (2020) Caida data - completed datasets. UC San Diego. <https://www.caida.org/catalog/datasets/completed-datasets/>
44. Fontugne R, Borgnat P, Abry P, Fukuda K (2010) Mawilab: Combining diverse anomaly detectors for automated anomaly labeling and performance benchmarking. In: *Proceedings of the 6th International Conference. Co-NEXT '10*. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/1921168.1921179>
45. Weisberg J. Argus - The all seeing System and Network Monitoring Software. <http://argus.tcp4me.com/>
46. Project Z. Bro Ids. Zeek. <https://old.zeek.org/manual/2.5.5/broids/index.html>
47. Claise EB (2004) Cisco Systems NetFlow Services Export Version 9. Internet Engineering Task Force. <https://www.ietf.org/rfc/rfc3954.txt>
48. Creech G, Hu J (2013) Generation of a new ids test dataset: Time to retire the kdd collection. In: 2013 IEEE Wireless Commun Netw Conf (WCNC), pp 4487–4492. <https://doi.org/10.1109/WCNC.2013.6555301>
49. Chou D, Jiang M (2021) A survey on data-driven network intrusion detection. *ACM Comput Surv* 54(9):1–36. <https://doi.org/10.1145/3472753>
50. Google: Iman Sharafaldin from Google Scholar. Google Scholar. https://scholar.google.com/citations?user=NGOL_BwAAAAJ
51. ROSAY A, CARLIER F, CHEVAL E, LEROUX P (2022) From cic-ids2017 to lycos-ids2017: A corrected dataset for better performance. In: *IEEE/WIC/ACM International Conference on Web Intelligence and Intelligent Agent Technology. WI-IAT '21*, pp 570–575. Association for Computing Machinery, New York, NY, USA . <https://doi.org/10.1145/3486622.3493973>
52. Liu L, Engelen G, Lynar T, Essam D, Joosen W (2022) Error prevalence in nids datasets: A case study on cic-ids-2017 and cse-cic-ids-2018. In: 2022 IEEE Conference on Communications and Network Security (CNS), pp 254–262. <https://doi.org/10.1109/CNS56114.2022.9947235>
53. Lanvin M, Gimenez P-F, Han Y, Majorczyk F, Mé L, Totel E (2022) Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes, pp 1–16. HAL. <https://hal.science/hal-03775466>
54. New Brunswick U (2017) IDS 2017 | Datasets | Research | Canadian Institute for Cybersecurity | UNB — unb.ca. <https://www.unb.ca/cic/datasets/ids-2017.html>. [Accessed 24-10-2023]
55. Ring M, Wunderlich S, Grödl D, Landes D, Hotho A (2017) Flow-based benchmark data sets for intrusion detection. In: *Proceedings of the 16th European Conference on Cyber Warfare and Security (ECCWS)*, pp 361–369. ACPI, ???
56. Moustafa N (2019) The Bot-IoT dataset. *IEEE Dataport* . <https://doi.org/10.21227/fesz-dm97> . <https://dx.doi.org/10.21227/fesz-dm97>
57. Project Z . The Zeek Network Security Monitor. Zeek Project. <https://zeek.org/>
58. Moustafa N (2019) The Bot-IoT dataset. *IEEE Dataport*. <https://doi.org/10.21227/r7v2-x988> . <https://dx.doi.org/10.21227/r7v2-x988>
59. Akgun D, Hizal S, Cavusoglu U (2022) A new DDoS attacks intrusion detection model based on deep learning for cybersecurity. *Comput Secur* 118:102748. <https://doi.org/10.1016/j.cose.2022.102748>
60. García S, Parmisano A, Erquiaga MJ (2020) IoT-23: A labeled dataset with malicious and benign IoT network traffic. Zenodo. More details here <https://www.stratosphereips.org/datasets-iot23>. <https://doi.org/10.5281/zenodo.4743746>
61. Chatzoglou E, Kouliaridis V, Kambourakis G, Karopoulos G, Gritzalis S (2023) A hands-on gaze on http/3 security through the lens of http/2 and a public dataset. *Comput Secur* 125:103051. <https://doi.org/10.1016/j.cose.2022.103051>
62. Neto ECP, Taslimasa H, Dadkhah S, Iqbal S, Xiong P, Rahman T, Ghorbani AA (2024) Ciciov2024: Advancing realistic ids

- approaches against dos and spoofing attack in iov can bus. *Internet of Things* 26:101209. <https://doi.org/10.1016/j.iot.2024.101209>
63. Haider W (2023) Next-Generation Intrusion Detection System-Dataset (NGIDS-DS). UNSW Sydney
 64. Garcia S, Grill M, Stiborek J, Zunino A (2014) An empirical comparison of botnet detection methods. *Comput Secur* 45:100–123
 65. Rojas JS, Pekar A, Rendon A, Corrales JC (2020) Smart user consumption profiling: Incremental learning-based ott service degradation. *IEEE Access* 8:207426–207442. <https://doi.org/10.1109/ACCESS.2020.3037971>
 66. Szumelda P, Orzechowski N, Rawski M, Janicki A (2022) Vhs-22 – a very heterogeneous set of network traffic data for threat detection. In: *Proceedings of the 2022 European Interdisciplinary Cybersecurity Conference. EICC '22*, pp 72–78. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3528580.3532843>
 67. Abdulganiyu OH, Ait Tchakoucht T, Saheed YK (2023) A systematic literature review for network intrusion detection system (ids). *Int J Inf Secur* 22(5):1125–1162
 68. Jain N, Jana PK (2023) A logically randomized forest algorithm for classification and regression problems. *Expert Syst Appl* 213:119225. <https://doi.org/10.1016/j.eswa.2022.119225>
 69. Yang B, Arshad MH, Zhao Q (2022) Packet-level and flow-level network intrusion detection based on reinforcement learning and adversarial training. *Algorithms* 15(12):453. <https://doi.org/10.3390/a15120453>
 70. Lopez-Martin M, Carro B, Sanchez-Esguevillas A (2020) Application of deep reinforcement learning to intrusion detection for supervised problems. *Expert Syst Appl* 141:112963. <https://doi.org/10.1016/j.eswa.2019.112963>
 71. Alrashdi I, Alqazzaz A, Aloufi E, Alharthi R, Zohdy M, Ming H (2019) Ad-iot: Anomaly detection of iot cyberattacks in smart city using machine learning. In: *2019 IEEE 9th Annual Computing and Communication Workshop and Conference (CCWC)*, pp 0305–0310. IEEE
 72. Pontes CF, De Souza MM, Gondim JJ, Bishop M, Marotta MA (2021) A new method for flow-based network intrusion detection using the inverse potts model. *IEEE Trans Netw Serv Manage* 18(2):1125–1136
 73. Souza M, Pontes C, Gondim J, Garcia LP, DaSilva L, Marotta MA (2021) A novel open set energy-based flow classifier for network intrusion detection. [arXiv:2109.11224](https://arxiv.org/abs/2109.11224)
 74. Zhang Z, Zhang Y, Guo D, Song M (2021) A scalable network intrusion detection system towards detecting, discovering, and learning unknown attacks. *Int J Mach Learn Cybern* 12(6):1649–1665. <https://doi.org/10.1007/s13042-020-01264-7>
 75. Liang S, Li Y, Srikant R (2018) Enhancing the reliability of out-of-distribution image detection in neural networks. In: *International Conference on Learning Representations (2018)*. <https://openreview.net/forum?id=H1VGkIxRZ>
 76. Armi L, Fekri-Ershad S (2019) Texture image analysis and texture classification methods-a review. [arXiv:1904.06554](https://arxiv.org/abs/1904.06554)
 77. Rogachev A, Melikhova E, Atamanov G (2021) Building artificial neural networks for NLP analysis and classification of target content. In: *Proceedings of the Conference on Current Problems of Our Time: the Relationship of Man and Society (CPT 2020)*. Atlantis Press, ??? <https://doi.org/10.2991/assehr.k.210225.058>
 78. Bui V, Pham TL, Nguyen H, Jang YM (2021) Data augmentation using generative adversarial network for automatic machine fault detection based on vibration signals. *Appl Sci* 11(5). <https://doi.org/10.3390/app11052166>
 79. Abedzadeh N, Jacobs M (2023) A survey in techniques for imbalanced intrusion detection system datasets. *Int J Comput Syst Eng* 17(1):9–18
 80. Basnet RB, Shash R, Johnson C, Walgren L, Doleck T (2019) Towards detecting and classifying network intrusion traffic using deep learning frameworks. *J Internet Serv Inf Secur* 9:1–17
 81. Belarbi O, Khan A, Camelli P, Spyridopoulos T (2022) An intrusion detection system based on deep belief networks. In: Su C, Sakurai K, Liu F (eds) *Sci Cyber Secur*. Springer, Cham, pp 377–392
 82. Chawla NV, Bowyer KW, Hall LO, Kegelmeyer WP (2002) Smote: Synthetic minority over-sampling technique. *J Artif Int Res* 16(1):321–357
 83. Khosla P, Teterwak P, Wang C, Sarna A, Tian Y, Isola P, Maschinot A, Liu C, Krishnan D (2020) Supervised contrastive learning. In: Larochelle H, Ranzato M, Hadsell R, Balcan MF, Lin H (eds) *Advances in Neural Information Processing Systems*, vol 33, pp 18661–18673. Curran Associates, Inc., ???
 84. Li T, Cao P, Yuan Y, Fan L, Yang Y, Feris RS, Indyk P, Katabi D (2021) Targeted supervised contrastive learning for long-tailed recognition. *2022 IEEE/CVF Conf Comput Vis Pattern Recogn (CVPR)*, 6908–6918
 85. Gao T, Yao X, Chen D (2021) SimCSE: Simple contrastive learning of sentence embeddings. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp 6894–6910. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic. <https://doi.org/10.18653/v1/2021.emnlp-main.552>
 86. Hinton GE, Salakhutdinov RR (2006) Reducing the dimensionality of data with neural networks. *Science* 313(5786):504–507. <https://doi.org/10.1126/science.1127647> <https://www.science.org/doi/pdf/10.1126/science.1127647>
 87. Liu L, Wang P, Ruan J, Lin J (2022) ConFlow: Contrast Network Flow Improving Class-Imbalanced Learning in Network Intrusion Detection. *Research Square*. <https://doi.org/10.21203/rs.3.rs-1572776/v1>
 88. Huang G, Liu Z, Weinberger, K.Q.: Densely connected convolutional networks. *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2261–2269
 89. Shafiq M, Gu Z (2022) Deep residual learning for image recognition: A survey. *Appl Sci* 12(18). <https://doi.org/10.3390/app12188972>
 90. NFStream: A flexible network data analysis framework. NFStream Developers. <https://www.nfstream.org/>
 91. Scarselli F, Gori M, Tsoi AC, Hagenbuchner M, Monfardini G (2009) The graph neural network model. *IEEE Trans Neural Netw* 20(1):61–80. <https://doi.org/10.1109/TNN.2008.2005605>
 92. Hamilton WL, Ying R, Leskovec J (2017) Inductive representation learning on large graphs. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems. NIPS'17*, pp 1025–1035. Curran Associates Inc., Red Hook, NY, USA
 93. Kilincer IF, Ertam F, Sengur A (2021) Machine learning methods for cyber security intrusion detection: Datasets and comparative study. *Comput Netw* 188:107840. <https://doi.org/10.1016/j.comnet.2021.107840>
 94. He K, Kim DD, Asghar MR (2023) Adversarial machine learning for network intrusion detection systems: A comprehensive survey. *IEEE Commun Surv Tutor* 25(1):538–566. <https://doi.org/10.1109/comst.2022.3233793>
 95. Zhang Y, Zhang H, Zhang B (2022) An effective ensemble automatic feature selection method for network intrusion detection. *Inf* 13(7):314. <https://doi.org/10.3390/info13070314>
 96. Ansari MS, Bartoš V, Lee B (2022) GRU-based deep learning approach for network intrusion alert prediction. *Future Gener Comput Syst* 128:235–247. <https://doi.org/10.1016/j.future.2021.09.040>
 97. Bozinovski S (2020) Reminder of the first paper on transfer learning in neural networks, 1976. *Inf* 44(3) <https://doi.org/10.31449/inf.v44i3.2828>

98. Lee J, Park K (2019) Ae-cgan model based high performance network intrusion detection system. *Appl Sci* 9(20). <https://doi.org/10.3390/app9204221>
99. Mvula PK, Branco P, Jourdan G-V, Viktor HL (2023) A systematic literature review of cyber-security data repositories and performance assessment metrics for semi-supervised learning. *Discover Data* 1(1) <https://doi.org/10.1007/s44248-023-00003-x>
100. Demšar J (2006) Statistical comparisons of classifiers over multiple data sets. *J Mach Learn Res* 7(1):1–30
101. Abdulganiyu OH, Tchakoucht TA, Saheed YK (2024) Towards an efficient model for network intrusion detection system (IDS): systematic literature review. *Wirel Netw* 30(1):453–482
102. Vitorino J, Praça I, Maia E (2023) SoK: Realistic adversarial attacks and defenses for intelligent network intrusion detection. *Comput Secur* 134(103433):103433
103. Verma J, Bhandari A, Singh G (2022) INIDS: SWOT analysis and TOWS inferences of state-of-the-art NIDS solutions for the development of intelligent network intrusion detection system. *Comput Commun* 195:227–247
104. García S, Grill M, Stiborek J, Zunino A (2014) An empirical comparison of botnet detection methods. *Comput Secur* 45:100–123
105. Verma J, Bhandari A, Singh G (2022) inids: Swot analysis and tows inferences of state-of-the-art nids solutions for the development of intelligent network intrusion detection system. *Comput Commun* 195:227–247
106. Verma J, Bhandari A, Singh G (2020) Review of existing data sets for network intrusion detection system. *Adv Math, Sci J* 9(6):3849–3854
107. Abdulganiyu OH, Tchakoucht TA, Saheed YK (2024) Towards an efficient model for network intrusion detection system (ids): systematic literature review. *Wirel Netw* 30(1):453–482
108. Nkongolo M, Deventer JP, Kasongo SM (2021) UGRansome1819: A novel dataset for anomaly detection and zero-day threats. *Inf (Basel)* 12(10):405
109. Gaudreault J-G, Branco P (2024) Empirical analysis of performance assessment for imbalanced classification. *Mach Learn*, 1–43

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Jose Carlos Mondragon¹  · Paula Branco³ · Guy-Vincent Jourdan³ · Andres Eduardo Gutierrez-Rodriguez^{2,4} · Rajesh Roshan Biswal¹

✉ Jose Carlos Mondragon
jose.c.mondragon@tec.mx

Paula Branco
pbranco@uottawa.ca

Guy-Vincent Jourdan
gjourdan@uottawa.ca

Andres Eduardo Gutierrez-Rodriguez
aegr82@tec.mx

Rajesh Roshan Biswal
rroshanb@tec.mx

¹ Tecnologico de Monterrey, School of Engineering and Sciences, Nuevo León, Nuevo León 64849, Mexico

² Tecnologico de Monterrey, Institute for the Future of Education, Nuevo León, Nuevo León 64849, Mexico

³ University of Ottawa, 800 King Edward Avenue, Ottawa, Ontario K1N6N5, Canada

⁴ Department, MAHLE Shared Services, Monterrey, Nuevo León 64650, Mexico

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval, sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com